

第9章 详细设计

本章导读

信息系统体系结构设计之后,需要着手详细设计。详细设计包括类与接口设计、功能逻辑设计、数据库设计和界面设计等工作。详细设计是系统实现的依据,需要考虑所有的设计细节,必须仔细认真。

主要知识点

■ 类与接口设计

■ 数据库设计

■ 功能逻辑设计

■ 界面设计

9.1 类与接口设计

9.1.1 概述

采用面向对象方法开发信息系统软件,类和对象是信息系统的基本构成要素,接口描述类对外所能提供的服务,因此,类与接口的设计是信息系统设计的一项重要工作。

OMG 在 2001 年提出了模型驱动架构(Model Driven Architecture, MDA)。MDA 把软件模型划分为计算无关模型(Computation Independent Model, CIM)、平台无关模型(Platform Independent Model, PIM)和平台相关模型(Platform Specific Model, PSM)三种类型。计算无关模型(CIM)用来建立与软件系统没有关系的业务领域的抽象模型,实际上就是业务领域模型。平台无关模型(PIM)面向业务领域的应用需求,与软件开发语言和环境无关。平台相关模型(PSM)则是考虑某种开发语言和环境的软件模型,通过 MDA 的转换规则,可以把 PIM 转换为 PSM。

在领域分析和系统设计中,都涉及类的设计与建模,但在不同阶段对类分析和建模的细致度和抽象程度并不同。在领域分析中建立的业务对象模型属于计算无关模型,它是概念层面的类模型。设计阶段将要建立平台无关类模型和平台相关类模型。

平台无关类模型简称为 PIM 类模型(Class Model Platform Independent Model),该模型独立于系统开发和运行平台。PIM 类模型由用来反映业务领域核心概念及系统静态逻辑结构,并且与实现平台没有关系的一组类构成,这组类及其关系构成了一个信息系统的核心



结构, 它既是系统的核心, 也是系统的基础。PIM 类模型来源于业务领域模型中的业务对象模型, 见图 9.1。

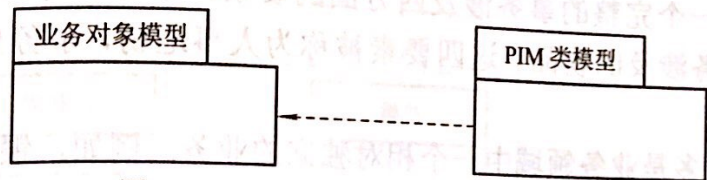


图 9.1 PIM 类模型来源于业务对象模型

平台相关类模型简称为 PSM 类模型(Class Model On Platform Specific Model), 该模型将作为软件编码的依据, 要给出类的所有设计细节, 包括类的属性、属性的初始值、属性的类型、操作、参数、返回类型、持久化、主键等。PSM 类模型中的类的描述还与编程语言有关。例如, 在定义属性类型时, Java 用 boolean 表示布尔型, 而 C# 则用 bool 表示布尔型。通过支持 MDA 的建模工具可以把 PIM 类模型自动转换为符合某种开发语言的平台相关 PSM 类模型。图 9.2 描述通过 Enterprise Architect 把 PIM 类模型转换为 PSM 类模型的过程, 表示基于 C# 和 Java 等环境的平台相关模型可以追溯到抽象平台无关模型。

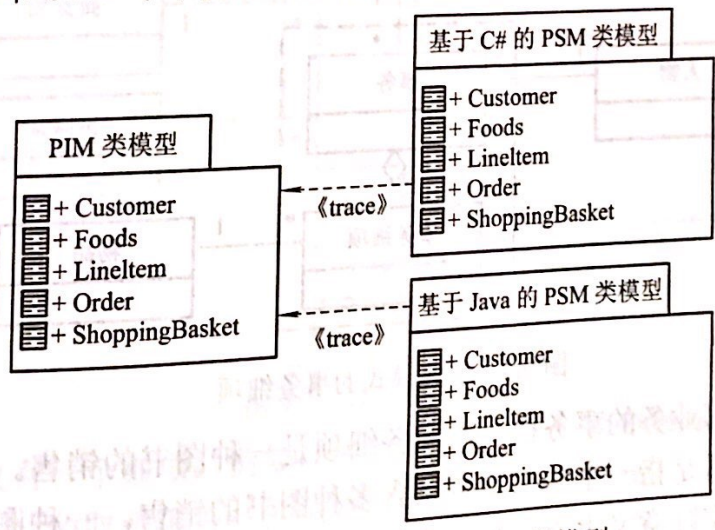


图 9.2 PIM 类模型转换为 PSM 类模型

1.2 事务模式

在介绍 PIM 类模型之前, 我们先讨论对建立 PIM 类模型有较大帮助的事务模式。事务模式(Transaction Pattern)由 OO 大师 Peter Coad 提出, 是用来反映业务领域中事务或结构的一种业务概念模式。事务模式框架见图 9.3。

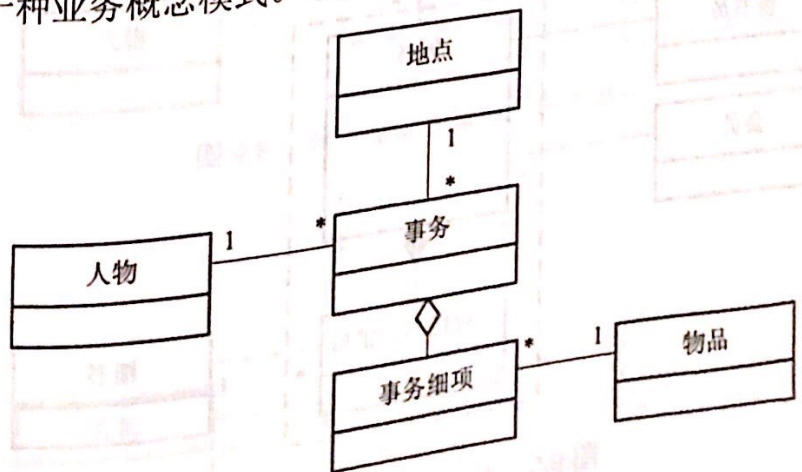


图 9.3 事务模式框架



1. 事务模式的四要素

在业务领域中，一个完整的事务涉及四方面的要素：事务本身、参与事务的人物、事务发生的地点、事务涉及的物品。这四要素被称为人事地物，事务处在四要素的中心位置。

(1) 事务。事务大多是业务领域中一个相对独立的业务。例如，银行的开户、存款、取款，超市的订货、入库、销售，宾馆的预订、入住、结账等业务都是事务。

一个事务常常会包括若干个事务细项。例如，一个订货事务会签订多个订购的货品，在该订货中，对每一种货品的订购就是一个事务细项。在书店的每一个售书事务中，会销售出多种图书，每一种图书的销售，就是一个事务细项。事务与其细项之间是组成关系，一个事务包括多个事务细项，见图 9.4。

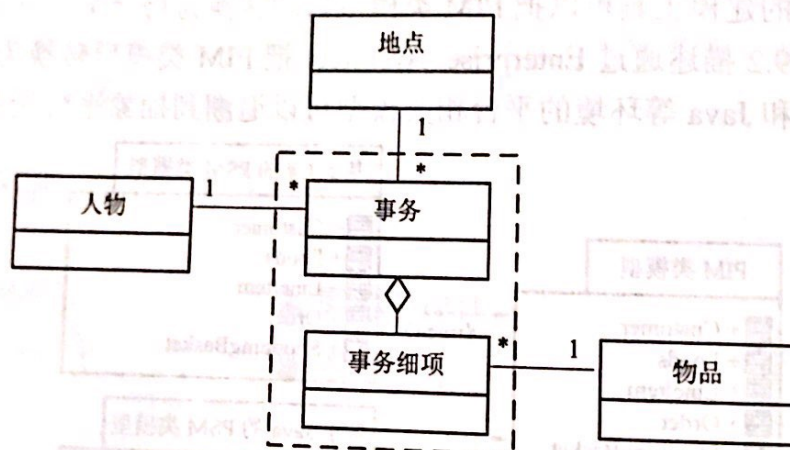


图 9.4 事务模式的事务细项

图 9.5 是书店售书业务的事务模式，事务细项是一种图书的销售。一种图书的销售是什么意思呢？它的本意是指一个售书事务包含多种图书的销售，一种图书的销售就是一个事务细项。从销售的实际意义上看，每一个售书事务，都会具体销售出多种图书，我们也可以把售书事务每销售出的一种图书作为一个事务细项，见图 9.6。同样道理，可以把订购的每一种货品作为订货事务的事务细项，见图 9.7。

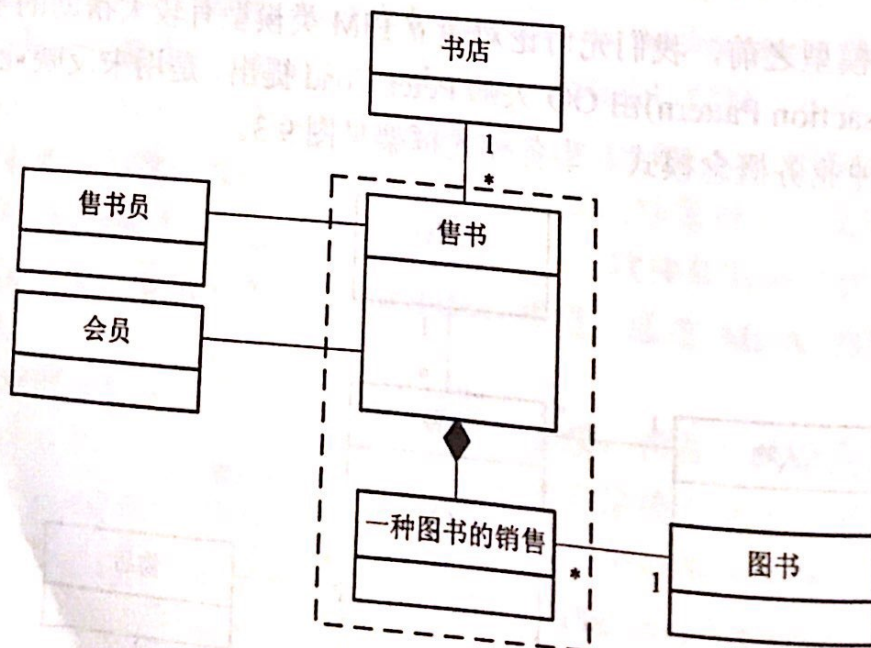


图 9.5 书店售书事务模式



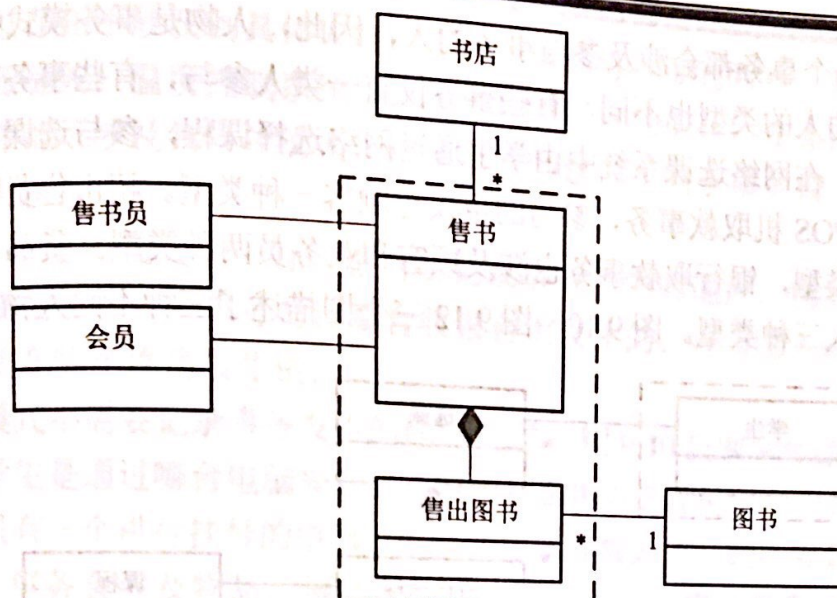


图 9.6 书店售书事务模式

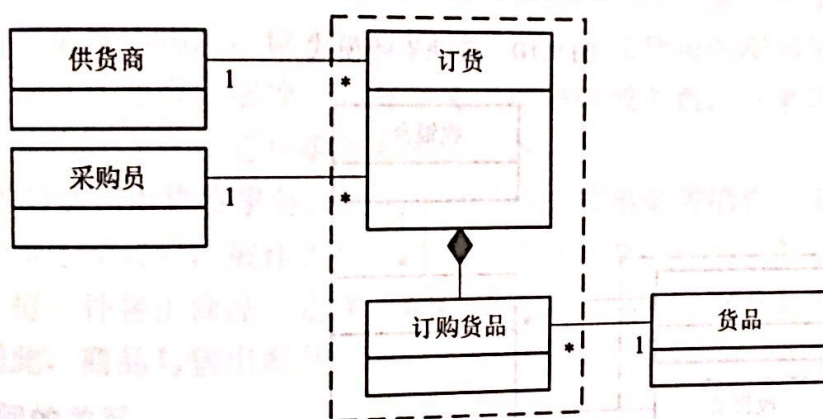


图 9.7 订货事务模式

有些事务可能仅有一个细项。例如，在银行系统中，一个取款事务只能取一笔款，两笔取款被视为两个不同的取款事务；医院的一个挂号事务也只能挂一个专家的号，同时挂两个专家的号，被认为是两个不同的挂号事务。仅有一个细项的事务就没有必要把事务细项独立出来。图 9.8 和图 9.9 分别描述了不分离事务细项的事务模式和举例。

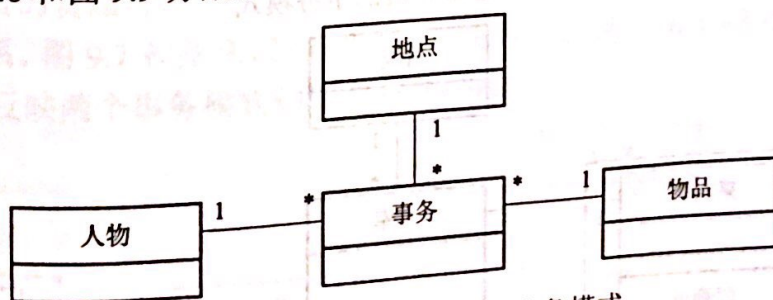


图 9.8 不分离事务细项的事务模式

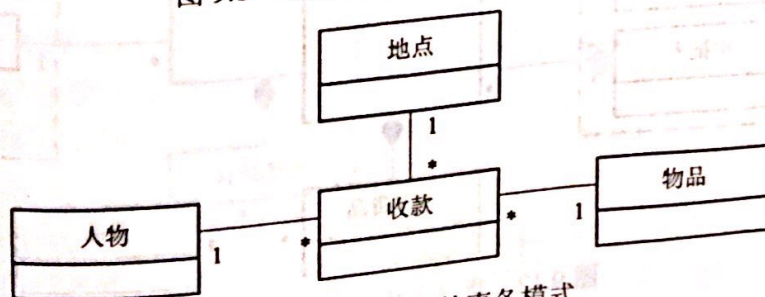


图 9.9 银行收款的事务模式



(2) 人物。每一个事务都会涉及参与事务的人，因此，人物是事务模式中的一个重要要素。不同事务参与的人的类型也不同，有些事务仅有一类人参与，有些事务可能会有两类或多类人参与。例如，在网络选课系统中由学生通过网络选择课程，参与选课事务的人只有学生一种类型。通过 POS 机取款事务，参与的也仅有顾客一种类型。超市售货事务则同时涉及顾客和收银员两种类型，银行取款事务也涉及顾客和业务员两种类型。货品入库则涉及采购员、库管员和审批人三种类型。图 9.10~图 9.12 三个图描述了三种不同人物类型的事务。

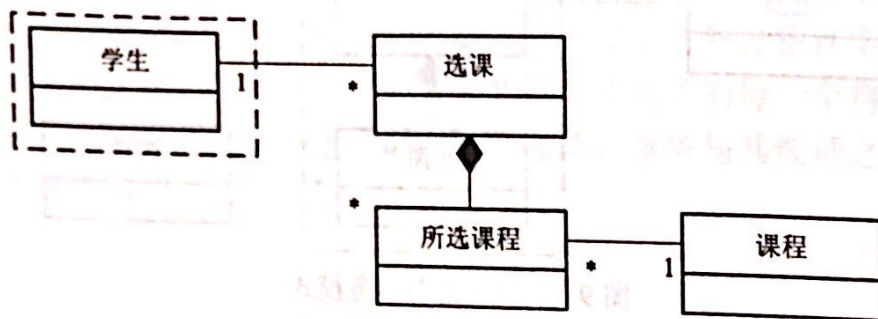


图 9.10 选课事务模式

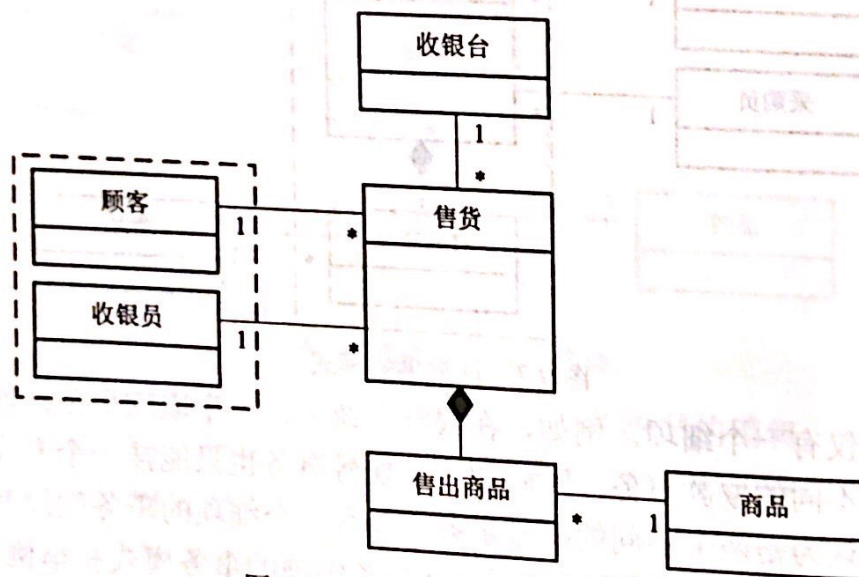


图 9.11 超市售货的事务模式

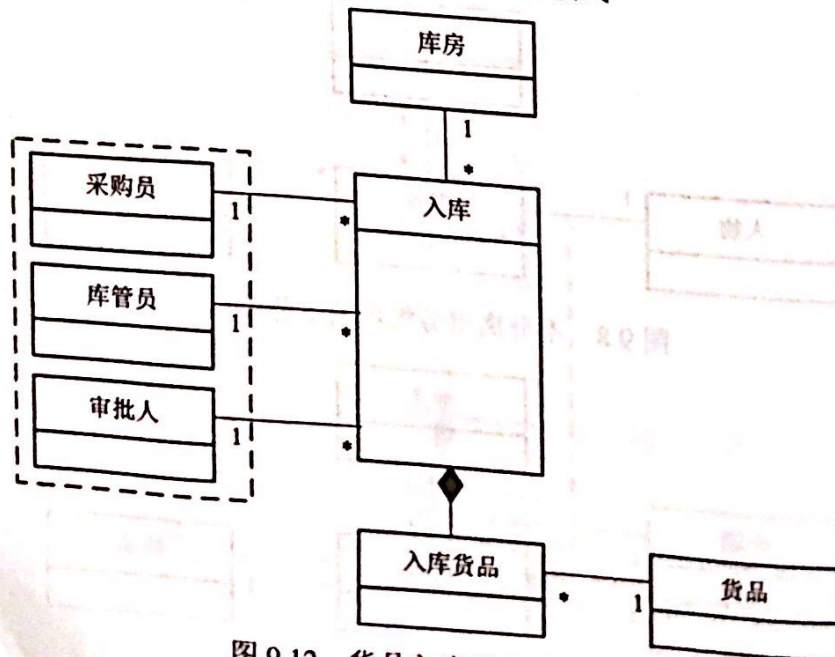


图 9.12 货品入库的事务模式



在现实中也存在不涉及到具体人物的事件。像路灯控制过程中,根据对外界光的感应决定对路灯开还是关;温度控制系统中,根据环境温度的变化,由系统来自动对温度进行调节,这些事件都不涉及到具体的人。这些事件的建模不同于事务模式,一般事务模式都离不开参与的人物。

(3) 地点。在事务模式中,有时也需要记录事务发生的地点。例如,在超市售货的事务模式中,需要记录每一个事务是在哪台收银台上发生的;在货品入库的事务模式中,需要记录本事务是在哪个库房发生的。

有些事务模式不需要记录事务发生的地点。例如,图9.10反映学生通过网络选课的事务,就不需要记录学生是通过哪台电脑来选课,或在什么地方选课的;一个职工医院只有一个挂号室,而且也只有有一个进行挂号的电脑,那么在该事务模式中,就不需要记录挂号的地点。

(4) 物品。事务会涉及物品。例如订货事务涉及货品,售书事务涉及图书,选课事务涉及课程等。事务涉及的物品有两种类型,一种是一般物品,像订货事务的货品,售书事务的图书;另一种是具体物品,像本次订购的货品,本次售出的图书等。在前面,我们把事务涉及的具体物品作为事务细项,它与事务之间是组成关系,下来我们需要确定一般物品在事务模式中的位置,以及它与事务要素的关系。

图9.11描述的是超市售货事务,售出商品是指本次事务所销售出去的商品,它属于具体物品,在售货事务模式中,被作为售货事务的事务细项;商品则作为一般物品,表示商品的一般特性。每一种售出商品一定属于某一种商品,而某一种商品可能会出现在不同的售货事务中,因此,商品与售出商品之间存在一对多的关联关系。

2. 事务之间的关系

在确定了业务中存在的各种事务模式之后,下面我们讨论不同事务之间可能存在的关系。事务之间存在顺序、并发和嵌套等关系。

(1) 顺序事务。顺序事务指事务之间是顺序关系,在前一事务完成后才能执行后一事。例如,在物资进销存系统中,订货和入库两个事务之间存在顺序关系,订货之后,才能入库,一次订货的货品可以一次入库,也可以分次入库,订货事务与入库事务之间存在一对多的关联关系。图9.7和图9.12分别描述订货和入库两个事务模式,在两图的基础上,图9.13给出能够反映两个事务模式顺序关系的类图。

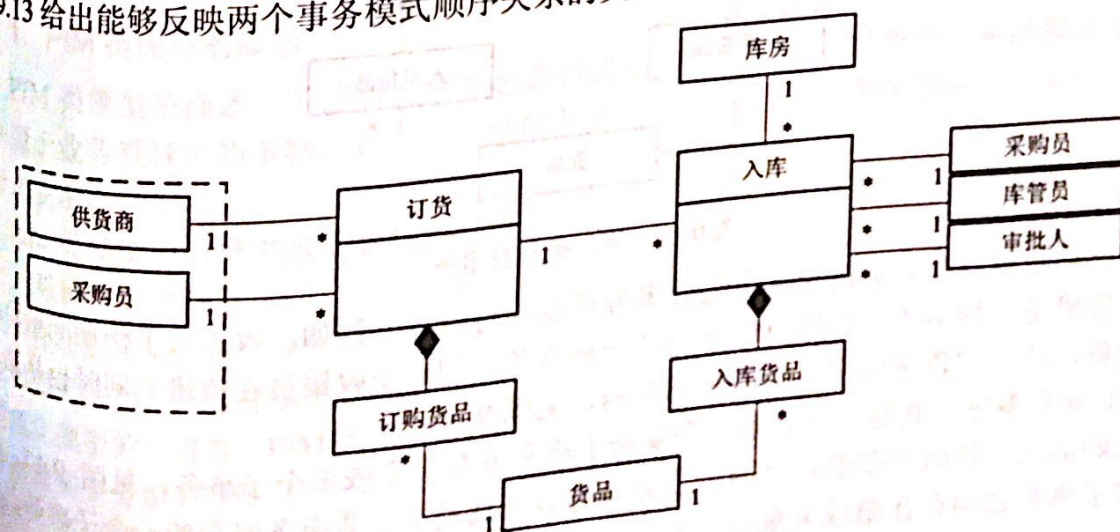


图9.13 事务的顺序关系



现实中，事务之间存在顺序关系的情况很多。在宾馆业务中，订房和入住两个事务之间是顺序关系，入住和结账也是顺序关系。银行的贷款和还款，网上购物和付款等事务都是顺序关系。

(2) 并发事务。并发事务是指可以同时发生的事务。事务之间存在并发关系有三类情况。第一类是两个事务之间没有任何关系，一个事务的发生不会影响到另外一个事务。例如图 9.14 中，学生的选课与就餐两个事务之间没有任何关系，这两个事务就属于并发事务。

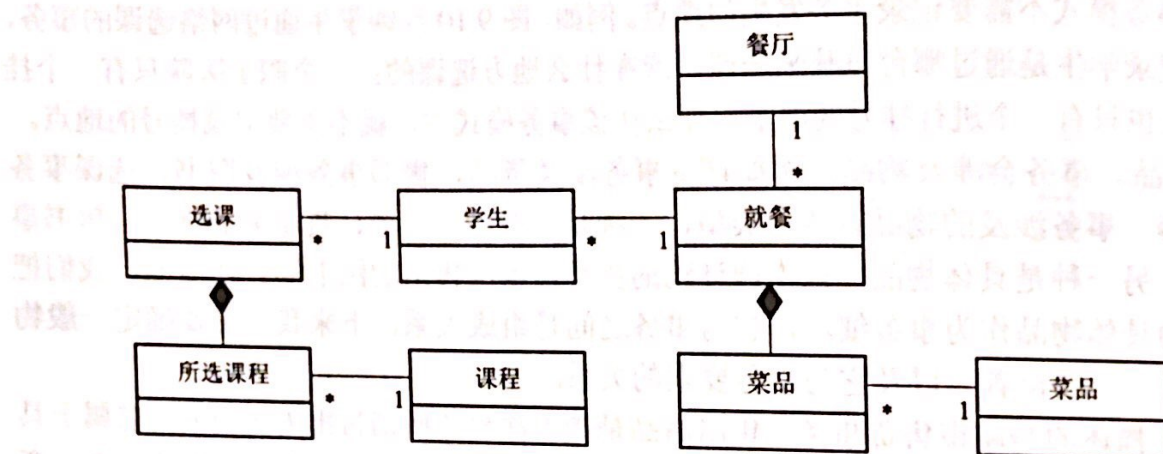


图 9.14 第一类并发事务

第二类是两个事务要素中的实例没有交集。例如，图 9.13 中，一批货品的订货与另外一个订货的入库事务就可以并发，见图 9.15。

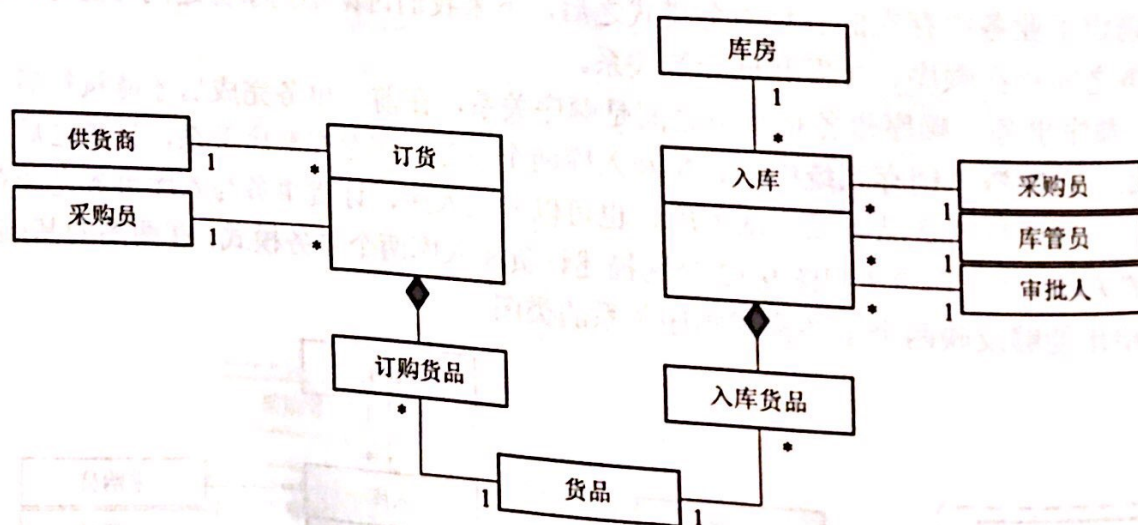


图 9.15 第二类并发事务

第三类是事务要素实例存在交集的事务可以同时发生。例如，收银员工作期间被称为售货业务，值班和售货两个事务就是并发的，见图 9.16。

(3) 嵌套事务。嵌套事务是指一个复杂主事务嵌套一个或多个子事务，见图 9.17。例如，采购员在订货时需要签订合同，签订合同事务就是被订货事务嵌套的一个子事务。主事务和子事务之间存在组成关系。



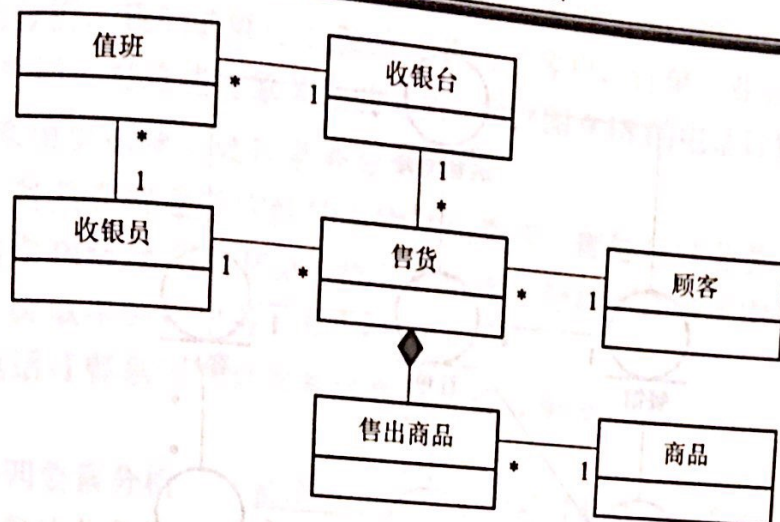


图 9.16 第三类并发事务

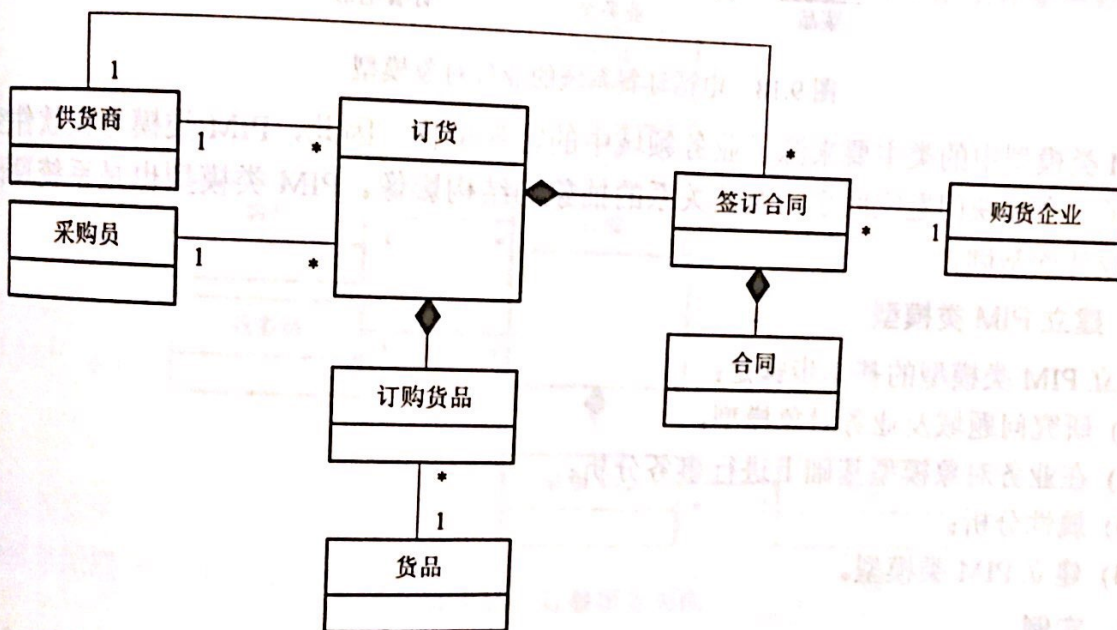


图 9.17 嵌套事务

9.1.3 PIM 类模型

1. PIM 类模型的概念

PIM 类模型是描述一个信息系统静态逻辑要素及其结构的一组类图。系统静态逻辑要素来源于业务领域中的事物，是信息系统的基本逻辑要素。PIM 类模型中的类采用类的常规形式表示。

PIM 类模型中的类来源于业务领域模型中的业务对象，但与业务对象并不完全对应。第一，两种模型的侧重不同。业务对象模型反映业务领域中存在的事物及其关系，在业务领域中存在的事物，就可能作为业务对象出现在业务对象模型中。PIM 类模型作为信息系统的逻辑要素结构，只选择那些系统需要的业务对象。第二，作为系统的逻辑要素结构必须考虑所建立模型的合理性。PIM 类模型需要在业务对象模型基础上进行深化分析，对于复杂的业务对象需要分解。另外，出于某些功能或性能的需要，可能还需要增加一些类。例如，“菜品种类”类（可参见图 9.42）在图 9.18 的业务对象模型中并不存在，但为了使得开发的电话订餐系统能够灵活适用于各种餐馆的菜品预订业务，把菜品种类独立成为一个类。



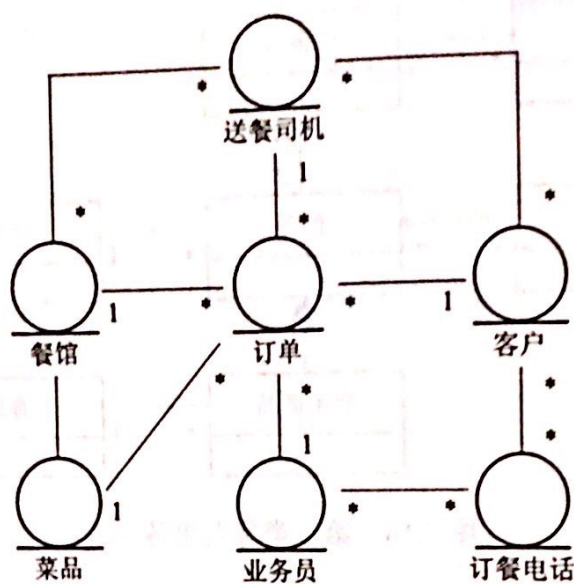


图 9.18 电话订餐系统的业务对象模型

PIM 类模型中的类主要来源于业务领域中的客观事物，因此，PIM 类模型在软件空间中建立了一个反映问题空间事物及其关系的抽象的结构影像。PIM 类模型也是系统设计和数据库设计的基础。

2. 建立 PIM 类模型

建立 PIM 类模型的基本步骤是：

- (1) 研究问题域及业务对象模型；
- (2) 在业务对象模型基础上进行事务分析；
- (3) 属性分析；
- (4) 建立 PIM 类模型。

3. 实例

下面以电话订餐系统和小型书店书务系统的 PIM 类模型为例，介绍建立 PIM 类模型的过程。

电话订餐系统

问题：赵丹大学毕业之后没有找工作，而是选择了自主创业之路。他在上大学期间就发现餐饮送货是一件具有广泛需求的服务，于是在自己创办的公司开展电话订餐业务。他和全市各知名餐馆建立联系，得到客户订单并负责将客户需要的饭菜送货上门。餐馆把订餐价格以批发价提供给他，他以零售价提供给客户。公司配置了几部订餐电话，业务员根据客户的电话订餐信息形成订单，并把订单信息转给一个餐馆，由餐馆准备饭菜。然后把餐馆地址、送餐时间、客户联系方式、送餐地址等信息提供给送餐司机，由司机负责送餐。有时客户可能打电话来，要求更改订单内容或取消订单，对这些信息需要及时通知餐馆。随着业务的发展，手工管理电话订餐业务已经不能满足业务需要，赵丹计划委托一家软件公司给他开发电话订餐系统。

1) 研究问题域及业务对象模型

业务对象模型是在领域分析中建立的反映问题领域事物及其关系的简单模型，该模型用来描述问题领域中所存在的客观事物及其关系，它是我们建立 PIM 类模型的基础。通过



对电话订餐业务的分析，我们能够提取餐馆、菜品、客户、订单、业务员、送餐司机和订餐电话等业务对象，并通过对这些对象关系的分析，建立图 9.18 的电话订餐系统业务对象模型。

2) 在业务对象模型基础上进行事务分析

事务模式是一种具有普遍适应性的事物结构模式，能够反映业务对象之间的相互联系和内在结构，是建立 PIM 类图的基础。事务分析重点分析所在领域中以事务为中心的人事地物的相互关系，提取事务是事务分析的关键。

经过分析，电话订餐系统有订餐和送餐两大核心事务。

★ 订餐事务

按照人事地物四要素分析，订餐事务的事务细项是订餐菜品，参与订餐的人物是客户和接听电话并登记订餐的业务员，地点是所订餐馆，物品是菜品。图 9.19 给出订餐事务的类图。

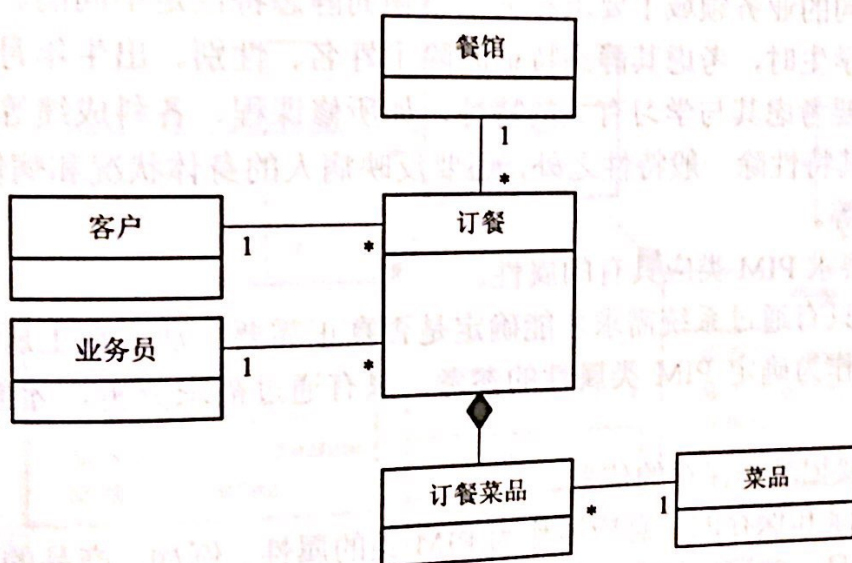


图 9.19 订餐事务类图

★ 送餐事务

送餐事务是订餐事务的后续事务。送餐事务的事务细项是订餐事务中所订的菜品，参与送餐事务的人物是客户和送餐司机，地点也是餐馆，因为司机需要从餐馆领到饭菜，再送给客户，送餐事务涉及的物品是菜品。图 9.20 是送餐事务的类图。

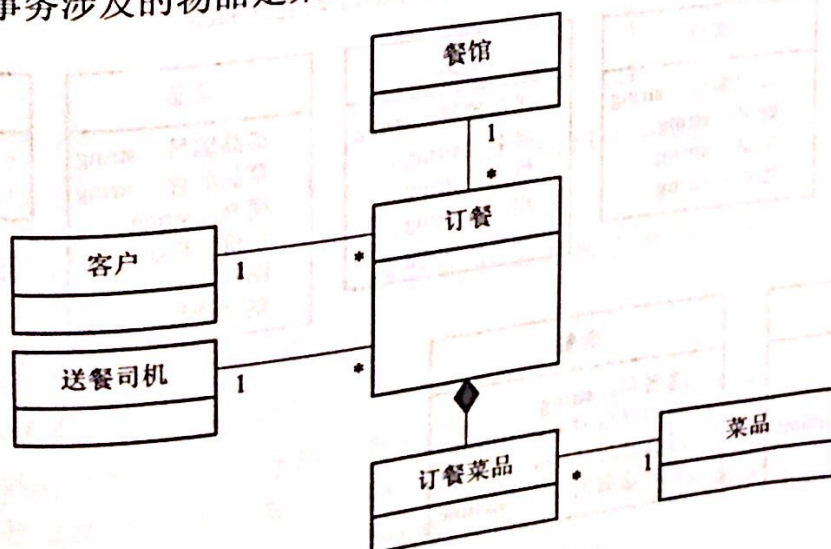


图 9.20 送餐事务类图



3) 属性设计

需要认真分析业务领域中的事物的性质,确定从业务对象来的 PIM 类的合适属性。属性分析的一般方法有下述几种:

(1) 从常理上看,业务对象所表示的事物有哪些静态特性?

绝大部分业务对象,尤其是实体型业务对象是用来直接反映业务领域中具体事物的,分析人员首先可以从常理上理解该业务对象所反映的事物应该有哪些静态特征,以便确定 PIM 类的属性。例如,学生的静态特性有学生的身份证号、姓名、性别、住址、邮政编码、电话、所学课程平均成绩、爱好、体重、身高、健康状况等。事物的静态特性可以作为 PIM 类的候选属性,但不能不加分析就直接作为 PIM 类的属性,因为信息系统也许并不需要某些特性。

(2) 在具体领域中 PIM 类所具有的属性。

同一事物在不同的业务领域中要求和突出事物的静态特性是不同的。例如,同一个人,当其在学校中作为学生时,考虑其静态特征时除了姓名、性别、出生年月、住址、电话等一般特性之外,还要考虑其与学习有关的特性,如所修课程、各科成绩等。但当此人在医院中作为病人时,其特性除一般特性之外,还要反映病人的身体状况和病情状况,如血压、脉搏、体温、体重等。

(3) 信息系统要求 PIM 类应具有的属性。

PIM 类的属性只有通过系统需求才能确定是否真正需要。从常理上所提出的属性和业务领域的属性只能作为确定 PIM 类属性的参考,只有通过需求分析,才能确定 PIM 类所需要的属性。

(4) PIM 类需要记录和保存的信息。

PIM 类需要记录和保存的信息应该作为 PIM 类的属性。例如,产品的生产量是需要记录和保存的产品信息,就可以把产品生产量作为产品类的一个属性。学生所修课程的成绩是需要保存的信息,学生所修课程成绩就可以作为学生类的属性。

图 9.21 给出初步确定的电话订餐系统 PIM 类模型中各个类的属性。需要说明的是,在业务对象模型中的业务员和送餐司机都是店员,因此设置店员类来代替业务员和送餐司机,并通过店员的岗位属性来区分店员的身份。另外,菜品类中包括菜品的全部信息,在订餐菜品中仅列出菜品编号和份数,没有给出菜品的其他信息。

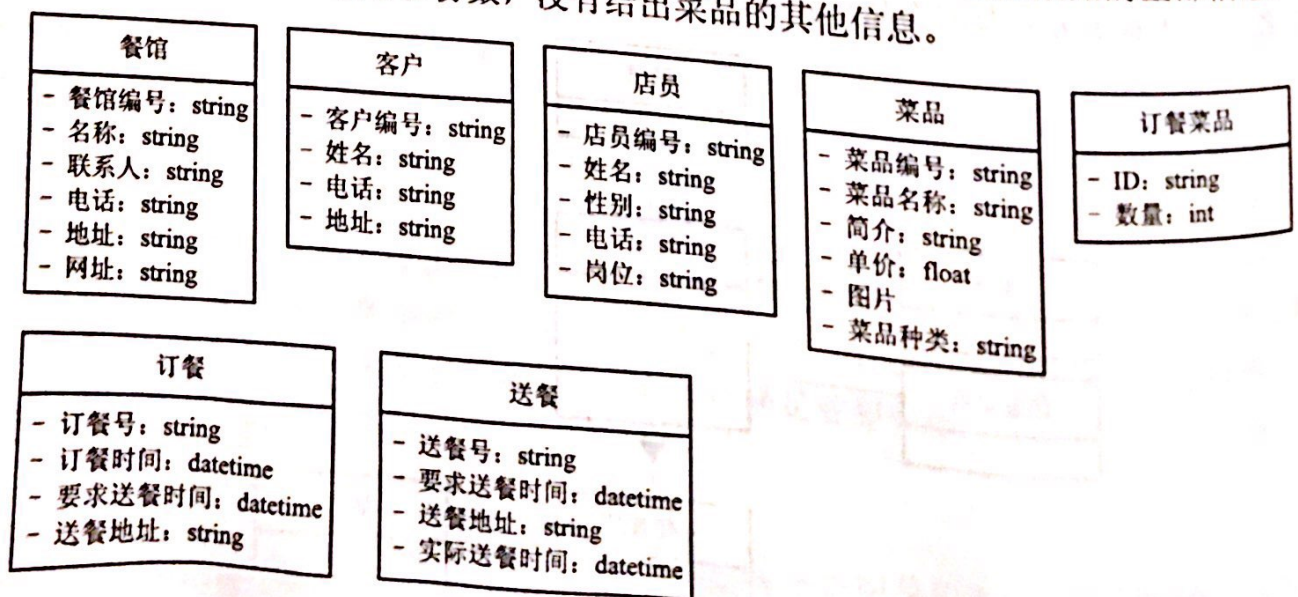


图 9.21 PIM 类的属性



4) 建立 PIM 类模型

在业务对象模型的基础上,通过各个事务及事务之间关系的分析,可以建立初步的 PIM 模型。经过对电话订餐系统事务分析,合并订餐和送餐两个事务,得出图 9.22 描述的初步的电话订餐系统的 PIM 类模型。

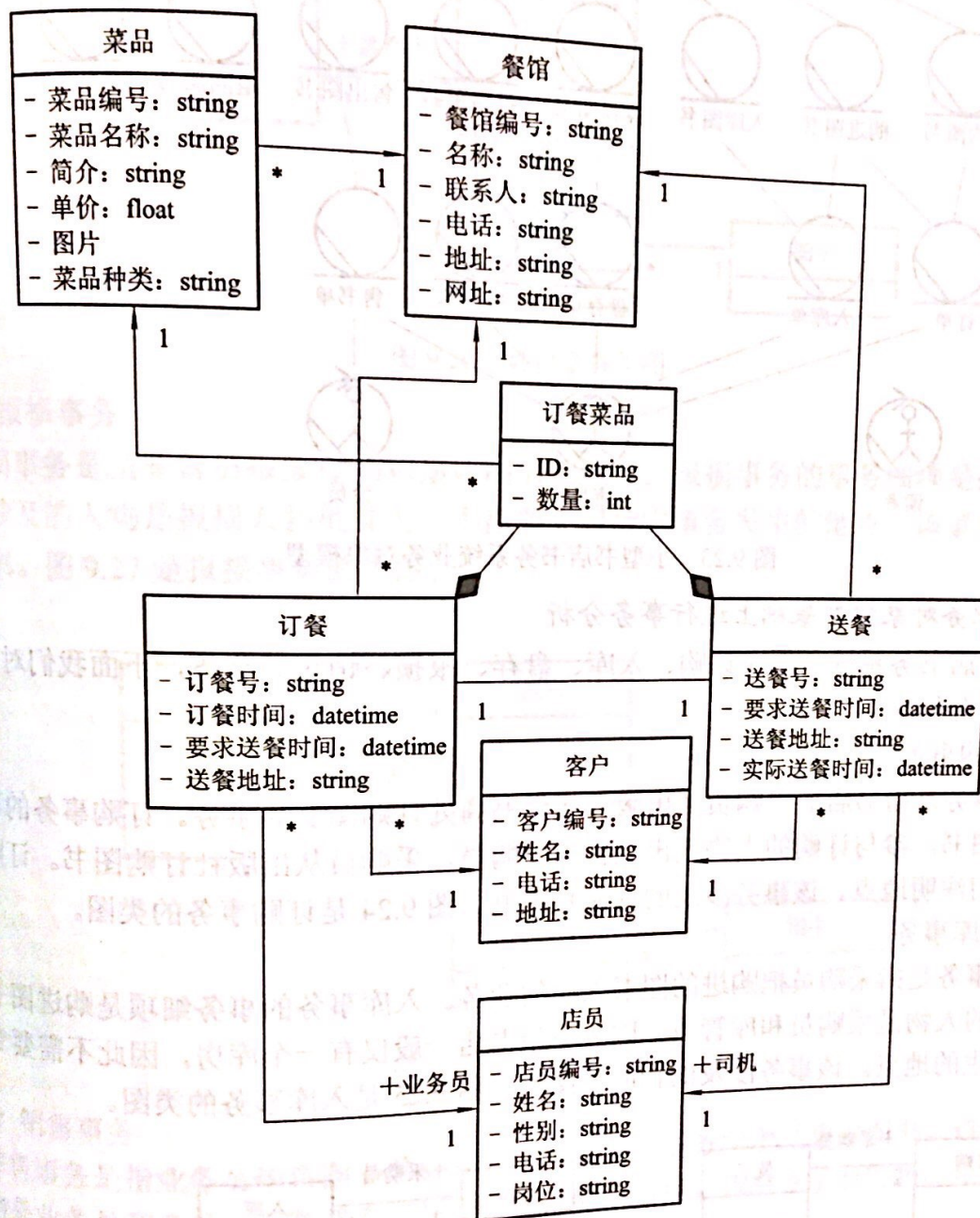


图 9.22 电话订餐系统初步的 PIM 类模型

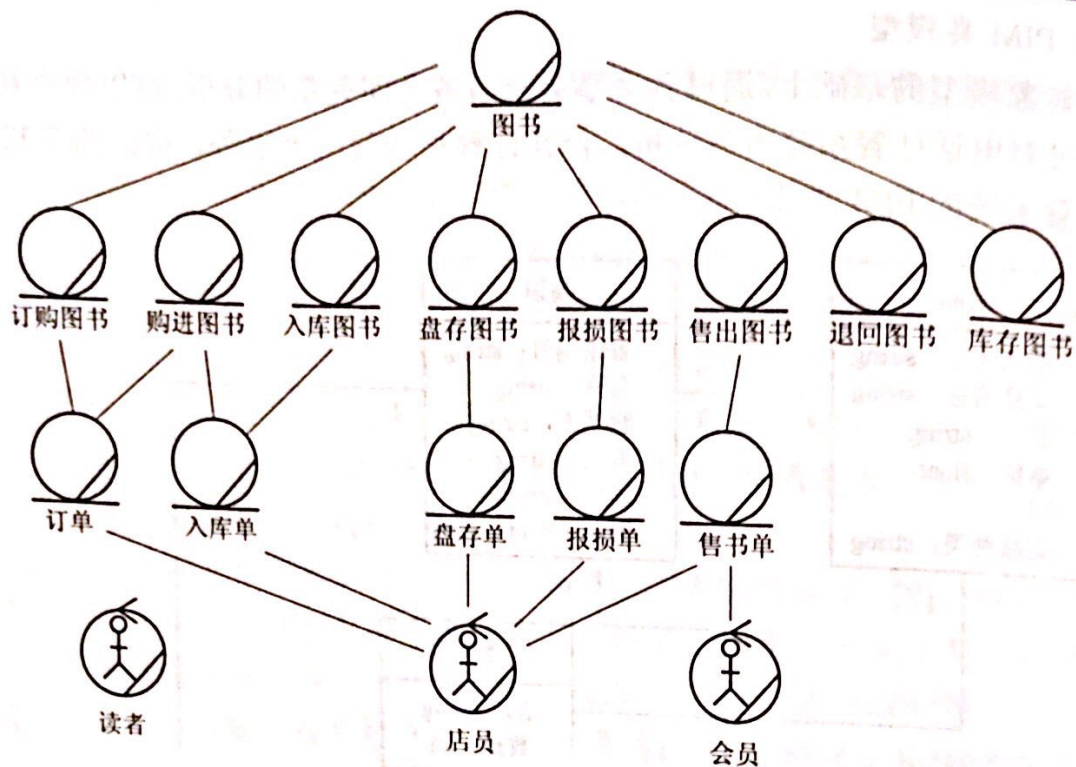
小型书店书务系统

在需求分析一章我们已经介绍了小型书店书务系统的基本功能,在此我们分析小型书店书务系统 PIM 类模型的提取过程。

1) 研究问题域及业务对象模型

通过对小型书店书务业务的分析,我们能够提取图书、订购图书、购进图书、入库图书、盘存图书、报损图书、售出图书、退回图书、库存图书、订单、入库单、盘存单、报单、售书单、读者、店员和会员等业务对象,并通过对这些对象关系的分析,建立图 9.23 所示的小型书店书务系统业务对象模型。





2) 在业务对象模型基础上进行事务分析

小型书店书务系统中存在订购、入库、盘存、报损、销售等事务，下面我们对这些事
务逐一进行分析。

★ 订购事务

订购事务是指书店的采购员从出版社或供书商处订购图书的事务。订购事务的事务细
项是订购图书，参与订购的人物是出版社和采购员，采购员从出版社订购图书。订购图书
不需要专门注明地点。该事务涉及的物品是图书。图 9.24 是订购事务的类图。

★ 入库事务

入库事务是指采购员把购进的图书入库的业务。入库事务的事务细项是购进图书。该
事务涉及的人物是采购员和库管员。因为小型书店一般仅有一个库房，因此不需要专门指
定事务发生的地点。该事务涉及的物品是图书。图 9.25 是入库事务的类图。

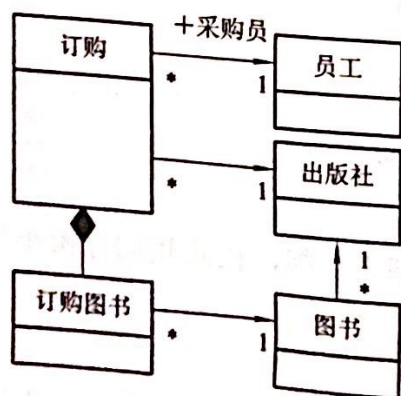


图 9.24 订购事务类图

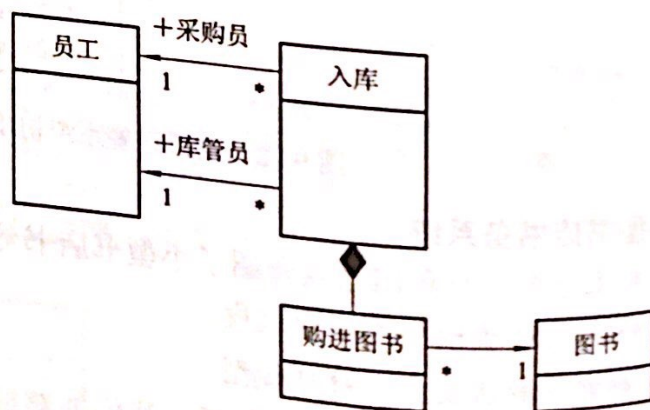


图 9.25 入库事务类图

★ 盘存事务

盘存事务是指库管员盘点图书的业务。盘存事务的事务细项是盘存图书。该事务涉及



的人物是盘存人和批准人。与入库事务相同，不需要专门指定事务发生的地点。该事务涉及的物品是图书。图 9.26 是盘存事务的类图。

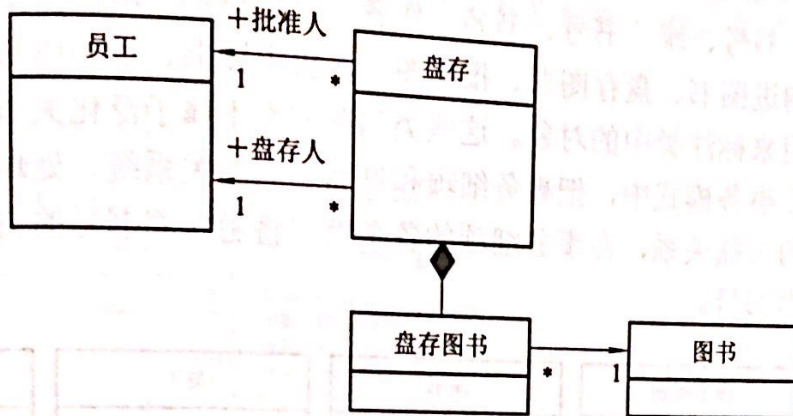


图 9.26 盘存事务类图

★ 报损事务

报损事务是指库管员报废过期和损坏图书的业务。报损事务的事务细项是报废图书。该事务涉及的人物是报损人和批准人。不需要专门指定事务发生的地点。该事务涉及的物品是图书。图 9.27 是报损事务的类图。

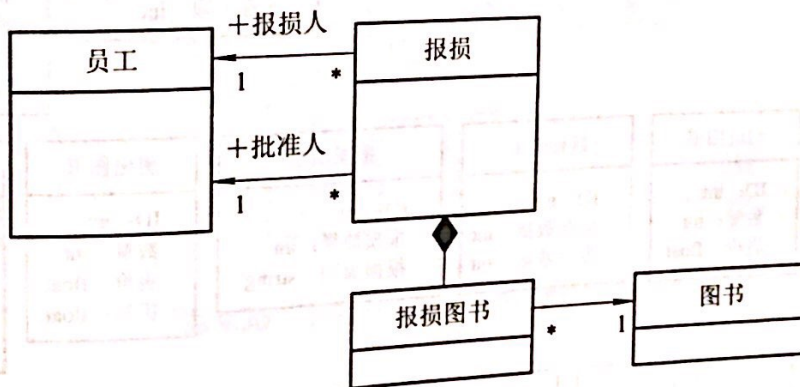


图 9.27 报损事务类图

★ 销售事务

销售事务是指业务员销售图书的业务。销售事务的事务细项是售出图书。该事务涉及的人物是业务员和会员。不需要专门指定事务发生的地点。该事务涉及的物品是图书。退回的图书也会与销售有关。图 9.28 是销售事务的类图。

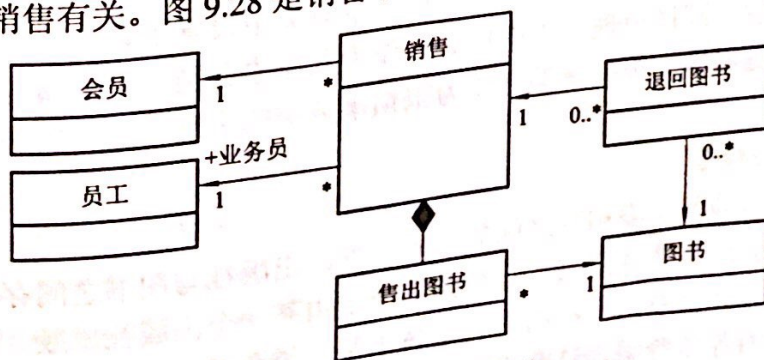


图 9.28 销售事务类图



3) 属性设计

图 9.29 给出初步确定的小型书店书务系统 PIM 类模型中各个类的属性。需要说明的是，图书类的属性，书号、统一书号、书名、作者、出版日期、版次、定价是图书的基本信息。订购图书、购进图书、盘存图书、报损图书、售出图书、退回图书和库存图书这几个类中设置的 ID 用来标注类中的对象。这些类和图书类本属于泛化关系，它们可以继承图书类的属性。但在事务模式中，把事务细项和物品之间的关系统一处理成由事务细项导航到物品的多对一的关联关系，在事务细项的各个类中将隐含着书号的关联属性，因此在此对这些类没有给出书号。

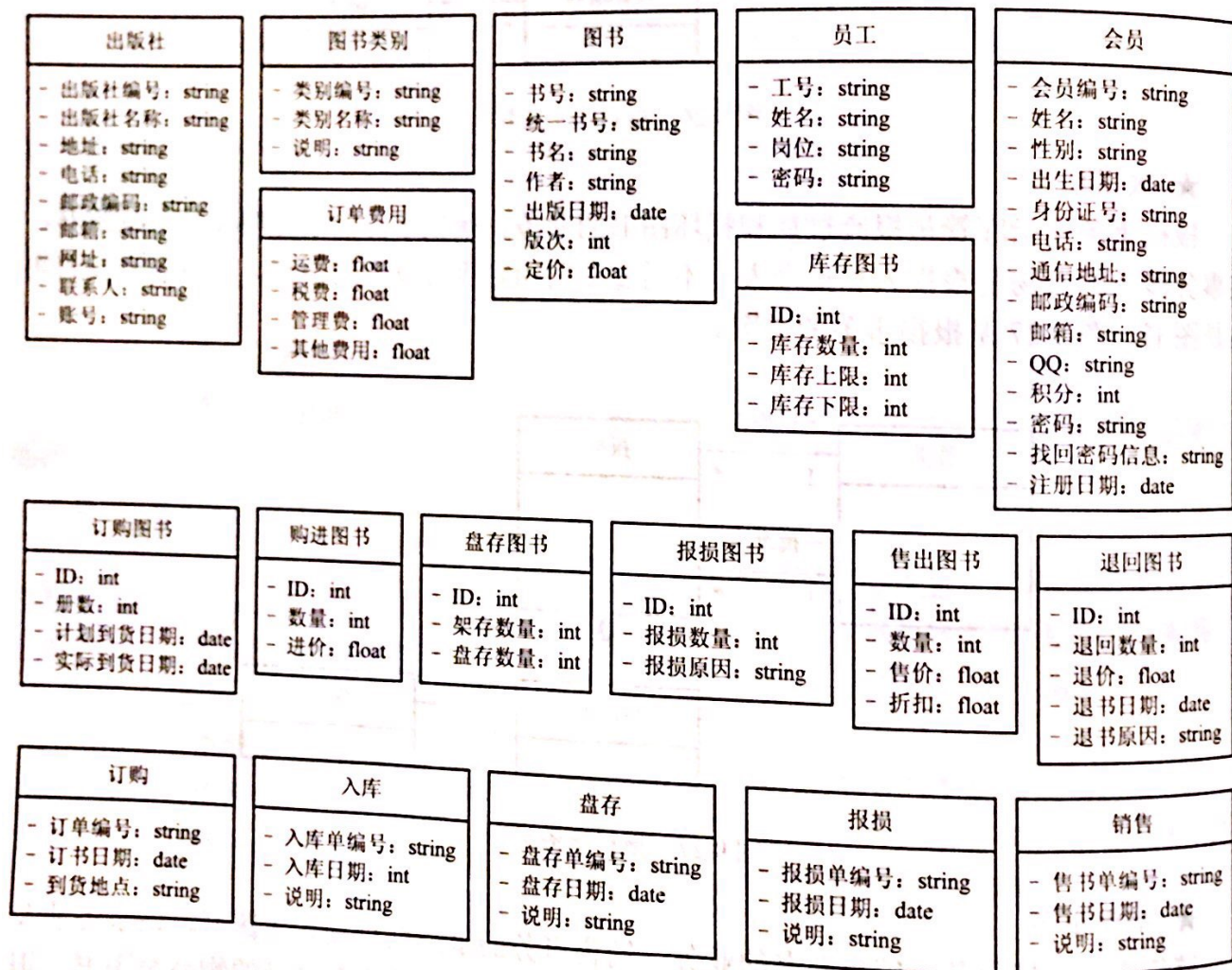


图 9.29 小型书店书务系统 PIM 类模型中类的属性

一个订购事务会产生一个订单，因此在订购事务类中设置了订单编号属性，该属性作为订购类的主键属性。同样道理，入库单编号作为入库事务类的主键属性，盘存单编号作为盘存事务类的主键属性，报损单编号作为报损事务类的主键属性，售书单编号作为售书事务的主键属性。

4) 建立 PIM 类模型

除了这些事务所涉及的类相互之间的关系之外，出版社与图书之间存在一对多的关联关系，一个出版社会出版多种图书，一种图书一定由某一个出版社出版。图书类别和图书之间存在一对多的关联关系，一种图书一定属于某一类图书。从订购事务中提取订单费用类，以反映该订单的各项费用，订购事务与订单费用属于组成关系。库存图书反映图书的



库存,它与图书之间是一一对一的关系。

入库是订购的后续事务,并且一次订购的图书可能会分多次入库。其他事务之间没有必然联系。经过分析给出图 9.30 所示的小型书店书务系统的 PIM 类模型。

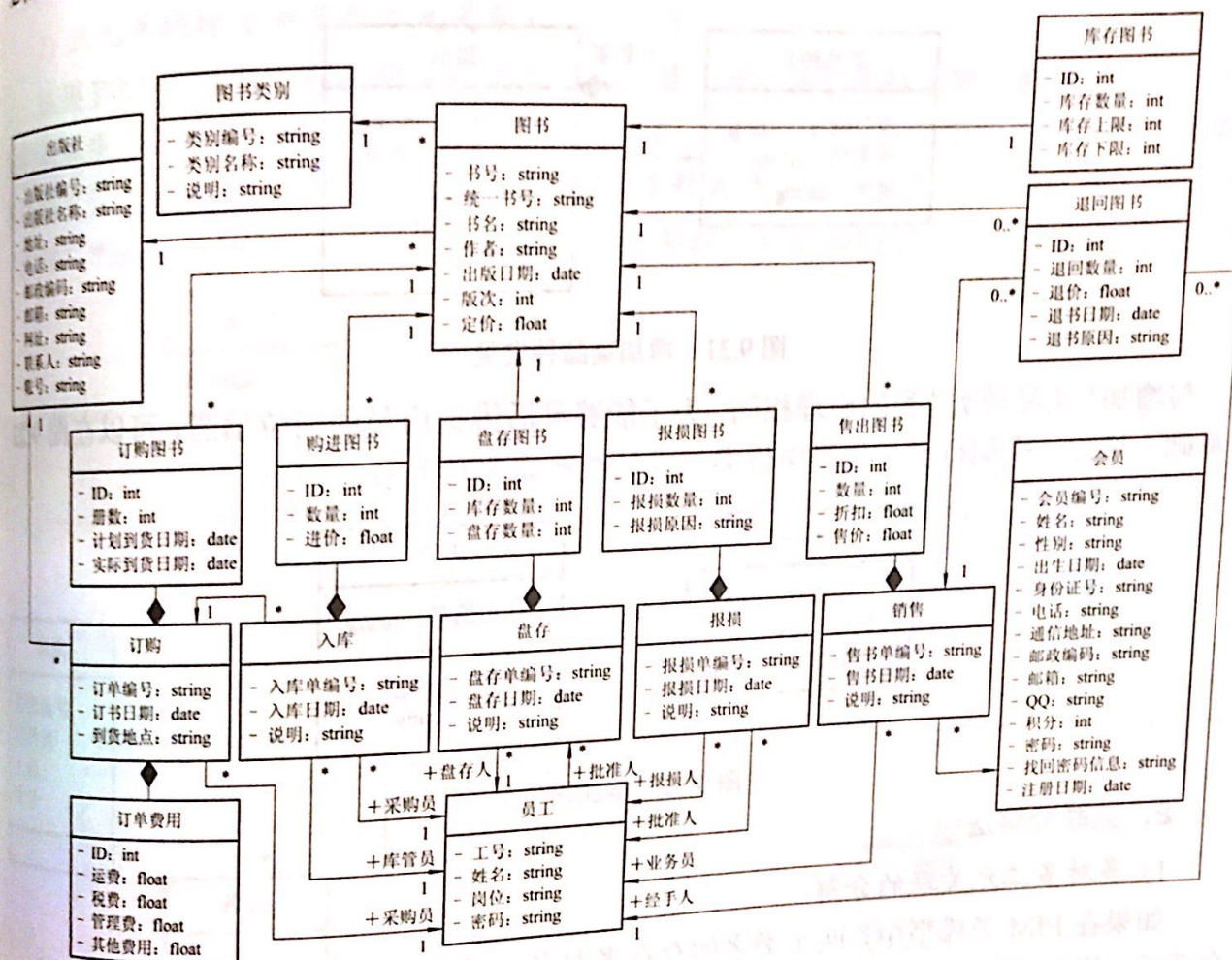


图 9.30 小型书店书务系统 PIM 类模型

9.1.4 PIM 类模型的优化

为了保证设计的合理性,对初步建立的 PIM 类模型需要进行优化。优化 PIM 类模型的原则是使 PIM 类模型能够准确地表示事物实体之间的自然结构,建立的类具有相对独立性、一致性和适中的规模。下面我们介绍几种常用的类模型优化方法。

1. 数据字典法

图 9.22 中的菜品类中有一个菜品种类属性,用这个属性来存放某个菜品所属的种类。对于经营川菜为主的餐馆而言,菜品种类的分类方法是确定的,而且种类数目不会太多。例如,经营川菜为主的餐馆中对菜品按照味型可以分为麻辣类、辛香类和咸鲜酸甜类三种类型。在系统增加菜品信息时,对菜品种类可以弹出一个下拉列表,给出已有的菜品种类的选项,让用户从中选择其中一个,这样既方便又不易出错。如果仅仅给出一个文本条让用户输入菜品种类,一方面输入速度慢,而且对于菜品种类不熟悉的人,可能在输入时出现错误。要系统能够给出菜品种类的选项,就需要在系统中预先存有该餐馆所经营的菜品的种类。为解决这个问题,可以采用数据字典法,专门设置一个“菜品种类”类,该类和菜品类保持关联关系,见图 9.31。“菜品种类”与“菜品”两个类之间是一对多的关联关系,由于在



“菜品”类中存在关联属性，可以把“菜品”中的菜品种类属性去掉。这样系统提供对菜品种类的维护功能，就可以适应不同订餐系统的菜品种类的要求了。

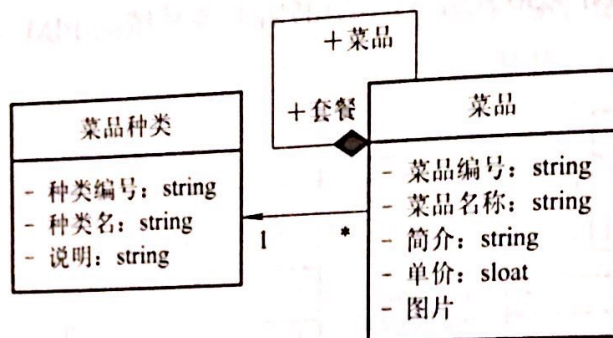


图 9.31 增加菜品种类类

与增加“菜品种类”类的原理相同，为了能够灵活维护店员的岗位信息，可以在图 9.22 的基础上增加一个岗位类，使得岗位类与店员类形成关联关系。

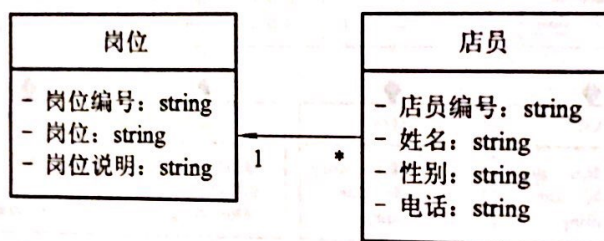


图 9.32 增加岗位类

2. 关联分解法

1) 多对多二元关联的分解

如果在 PIM 类模型中，两个类之间存在多对多二元关联，则需要在两个类之间增加一个新类，用这两个类中的关键属性作为这个类的属性。例如，图 9.33(a)中，“课程”和“教师”两个类之间存在多对多的关联关系，为了实现这两个类之间的双向关联，需要在它们之间增加一个“授课”类。由于“授课”类和“课程”及“教师”两个类之间存在多对一

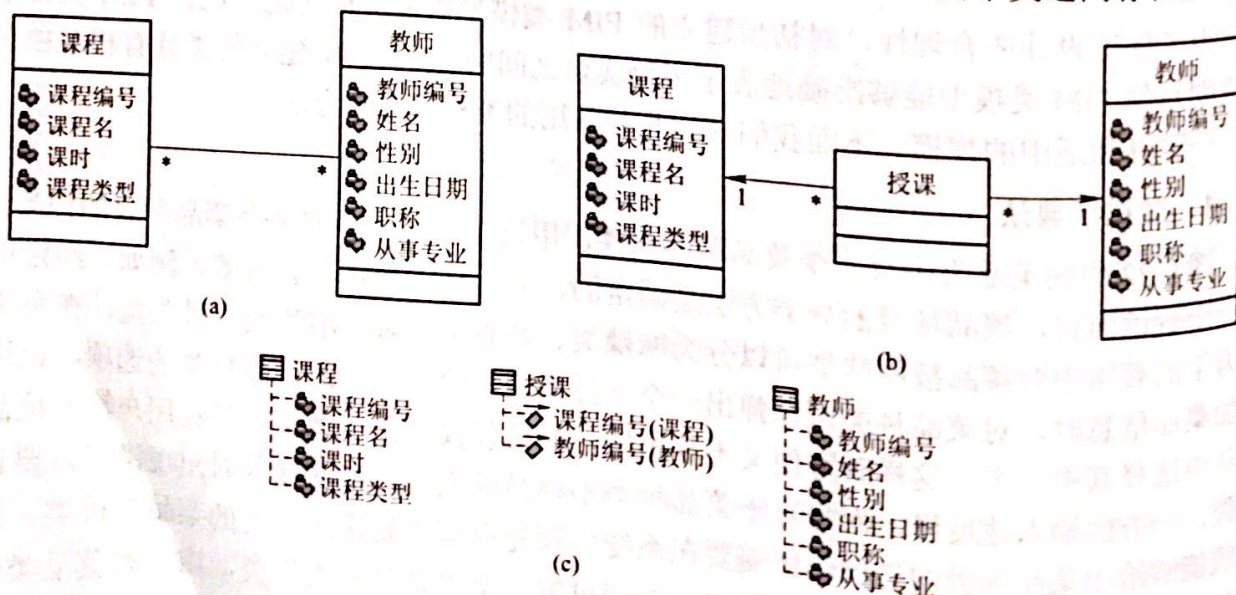


图 9.33 多对多关联的实现



的关联,因此在“授课”类中就隐含存在“课程”和“教师”两个类的关联属性“课程编号”和“教师编号”。这样把两个多对多的二元关联就转化成三个类中两两之间的一对多的关联,其导航关系分别是:从“授课”类到另外两个类,见图 9.33(b)和图 9.33(c)。

2) 三元关联转化为多个二元关联

如果 PIM 类模型中存在带关联类的三元关联,可以通过维持法来分解三元关联。该方法的基本思想是维持三个存在关联关系的类不变,由原来的关联类变为这三个类之间单向导航的二元关联关系,并且三个关联关系的多重性都是多对一的关系。例如,图 9.34(a)中,“教师”、“班级”、“课程”存在“授课”的三元关联,图 9.34(b)是对(a)中这种关系的优化。

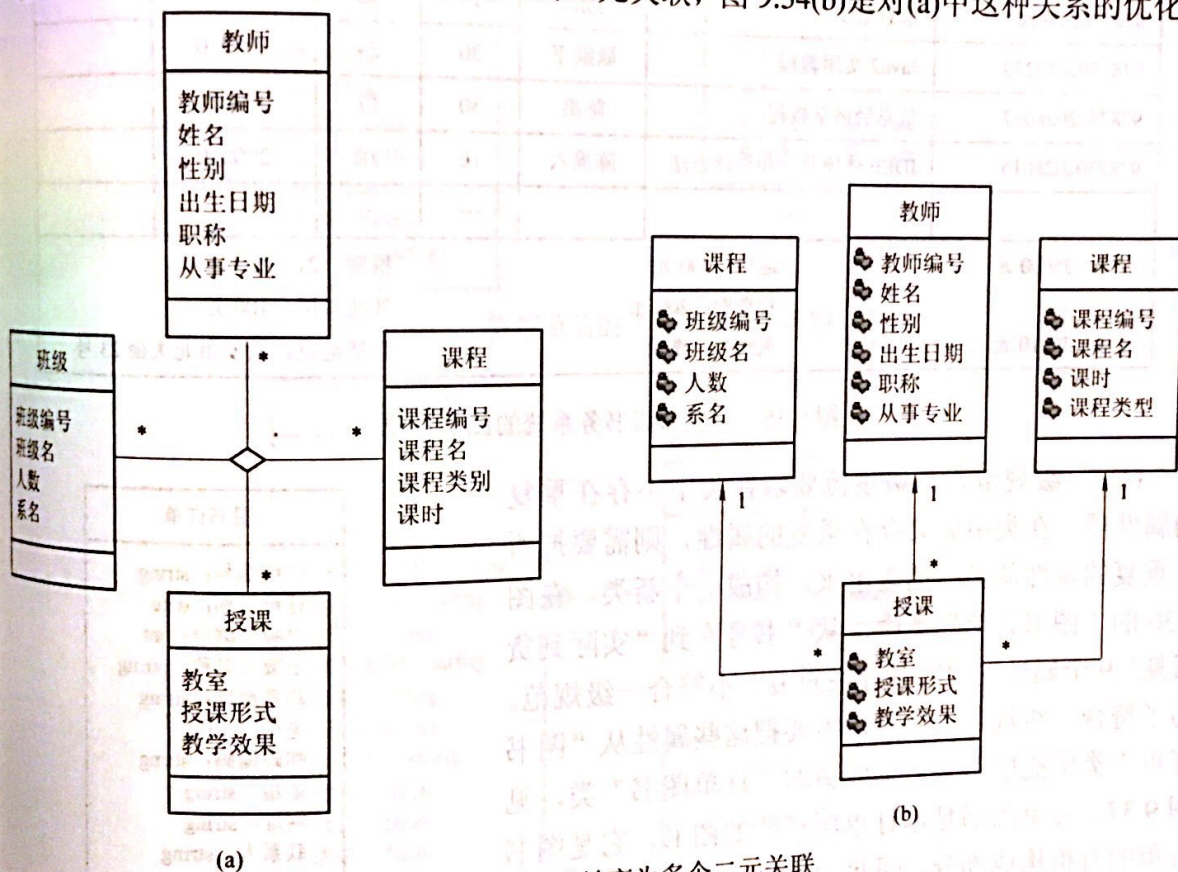


图 9.34 三元关联转变为多个二元关联

3. 规范化法

可以采用数据库设计中的规范化理论来对类进行优化。在数据库设计中,一般根据规范原则检查关系的优劣。如果一个关系符合范式规约,则可以说该关系是规范的;否则就需要对该关系进行优化处理,通过对关系进行分解,使其满足范式要求。实体类具有属性,关系也具有属性,因此,我们可以采用规范化方法来对实体类进行优化。关于规范化理论和方法读者可以参考有关数据库的书籍。在此,我们将以“图书订单”类的规范过程为例,讨论对实体类的优化过程。规范类满足三级规范要求。一级规范要求类中所有属性是不可再分的基本属性项;二级规范要求满足一级规范的基础上,类中不存在传递依赖关系。下面我们分四步对由图 9.35 “图书订单”所产生的“图书订单”类(见图 9.36)进行优化。首先,不加分析地建立一个如图 9.36 所示的“图书订单”类图。“图书订单”类完全是根据订单中包含的项目提取该类的属性的。



图书订单

订单编号: 100001
出版社编号: 0100111
电话: 010 87821671
地址: 北京清华大学学研大厦

填订日期: 2012.4
出版社名称: 清华大学出版社
账号: 83996587423
邮政编码: 100084

联系人: 赵正

书号	书名	作者	册数	订价	计划 到货日期	实际 到货日期
9787302038061	C 程序设计	谭浩强	100	26	200206	
9787302046507	信息组织	马张华	10	28	200205	
9787302047230	Java2 实用教程	耿跃平	30	29	200210	
9787302028017	信息经济学教程	陈禹	30	17	200210	
9787302028016	IDEF 建模分析和设计方法	陈禹六	10	28	200210	
	...		...	...		
合计: 35000 元		运费: 500 元		税费: 218 元		
		管理费: 200 元		其他费用: 100 元		
总计: 36010 元		采购员: 李明		到货地点: 西安市北大街 23 号		

图 9.35 小型书店书务系统的图书订单

(1) 一级规范。一级规范要求类中不存在重复的属性项。在类中如果存在重复的属性，则需要把所有重复的属性从类中抽取出来，构成一个新类。在图 9.36 的“图书订单”类中，从“书号”到“实际到货日期”9 个属性的内容都存在重复，不符合一级规范。为了符合一级规范的要求，需要把这些属性从“图书订单”类中提取出来，形成新的“订单图书”类，见图 9.37。订单图书是本订单所订购的图书，它是图书订单的有机组成部分，因此，“订单图书”类与“图书订单”类是组成关系，在一个订单中可以有多种订单图书。

(2) 二级规范。二级规范要求类中不存在部分依赖关系的属性，要把不完全依赖关键属性的非关键属性从类中提取出来。在图 9.37 中，“订单图书”类的主键属性是“订单编号”和“书号”，但是“ISBN”、“书名”、“作者”、“出版日期”、“版次”、“定价”等六个属性仅依赖“书号”属性，因此存在部分依赖关系。独立出一个单独的“图书”类，“订单图书”类与“图书”类形成关联关系，分解之后见图 9.38。

图书订单
- 订单编号: string
- 订单日期: date
- 出版社编号: int
- 出版社名称: string
- 联系地址: string
- 电话: string
- 邮政编码: string
- 邮箱: string
- 网址: string
- 联系人: string
- 账号: string
- 书号: string
- ISBN: string
- 书名: string
- 作者: string
- 出版日期: date
- 版次: int
- 定价: float
- 计划到货日期: date
- 实际到货日期: date
- 册数: int
- 合计: float
- 运费: float
- 税费: float
- 管理费: float
- 其他费用: float
- 总计: float
- 采购员: string
- 到货地点: string

图 9.36 初步的“图书订单”类图



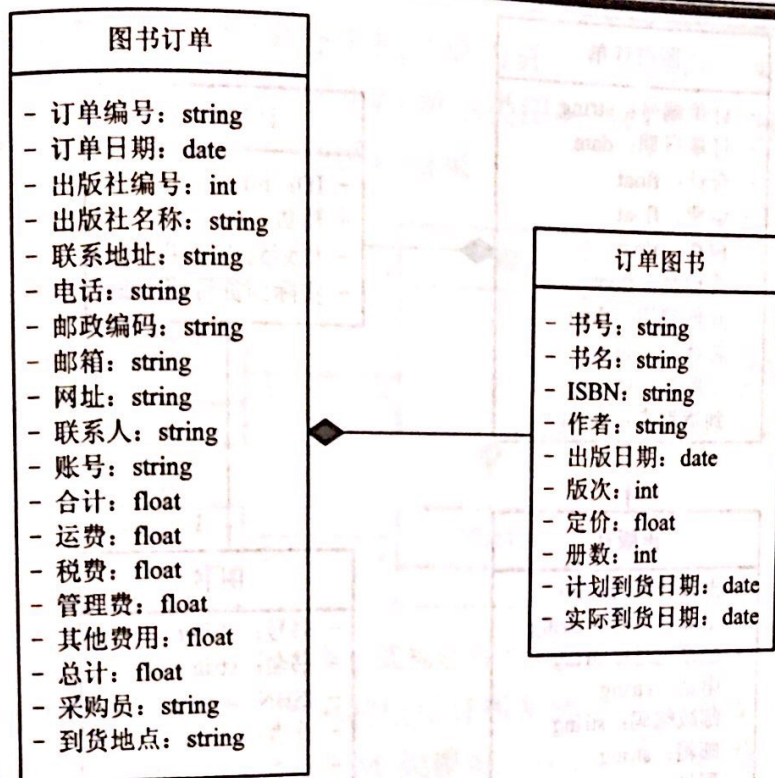


图 9.37 一级规范后的“图书订单”类图

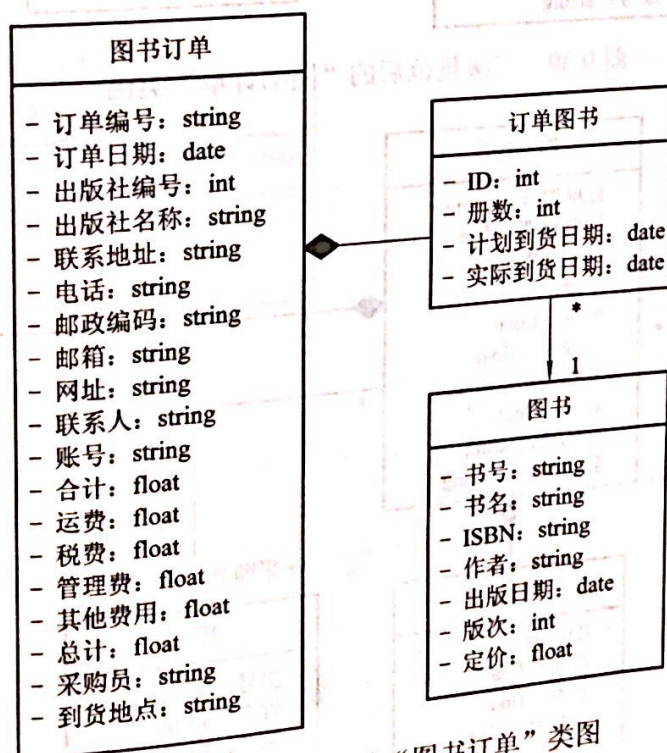


图 9.38 二级规范后的“图书订单”类图

(3) 三级规范。三级规范要求消除在类的属性中存在的传递依赖关系。在“图书订单”类中，“出版社编号”依赖“订单编号”，但是从“出版社名称”到“账号”8个属性仅依赖“出版社编号”，并不直接依赖“订单编号”，这是典型的传递依赖关系，需要消除。“图书”与“出版社”之间存在单向导航关系。规范之后的类图见图 9.39。

(4) 进一步优化。图 9.39 中“图书订单”的“合计”和“总计”两个属性属于派生属性，可以去掉。把几个费用属性独立出来形成一个新的“订单费用”类，作为“图书订单”类的部分类。“采购员”属于职工，可以生成一个“职工”类，采购员作为“图书订单”与“职工”类关联的角色名。这样优化之后的类图见图 9.40。



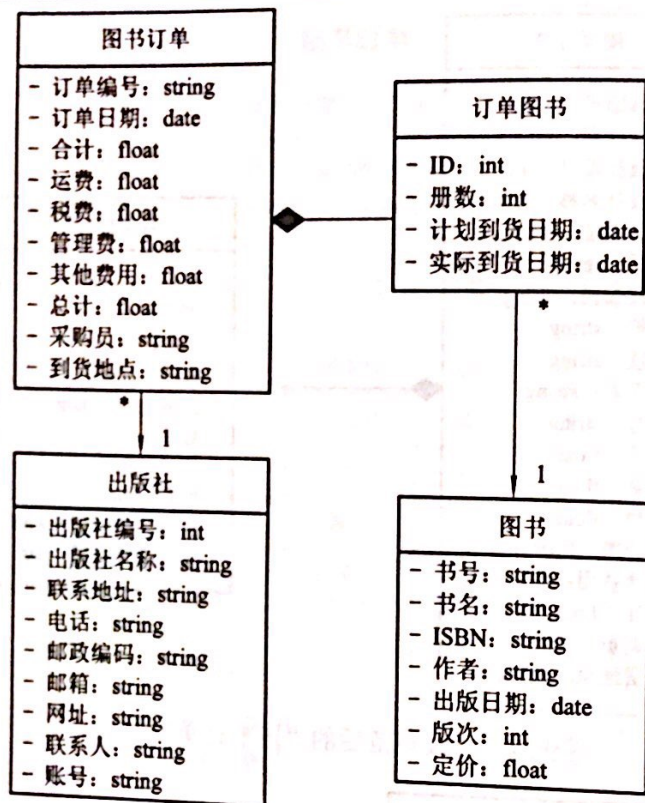


图 9.39 三级规范后的“图书订单”类图

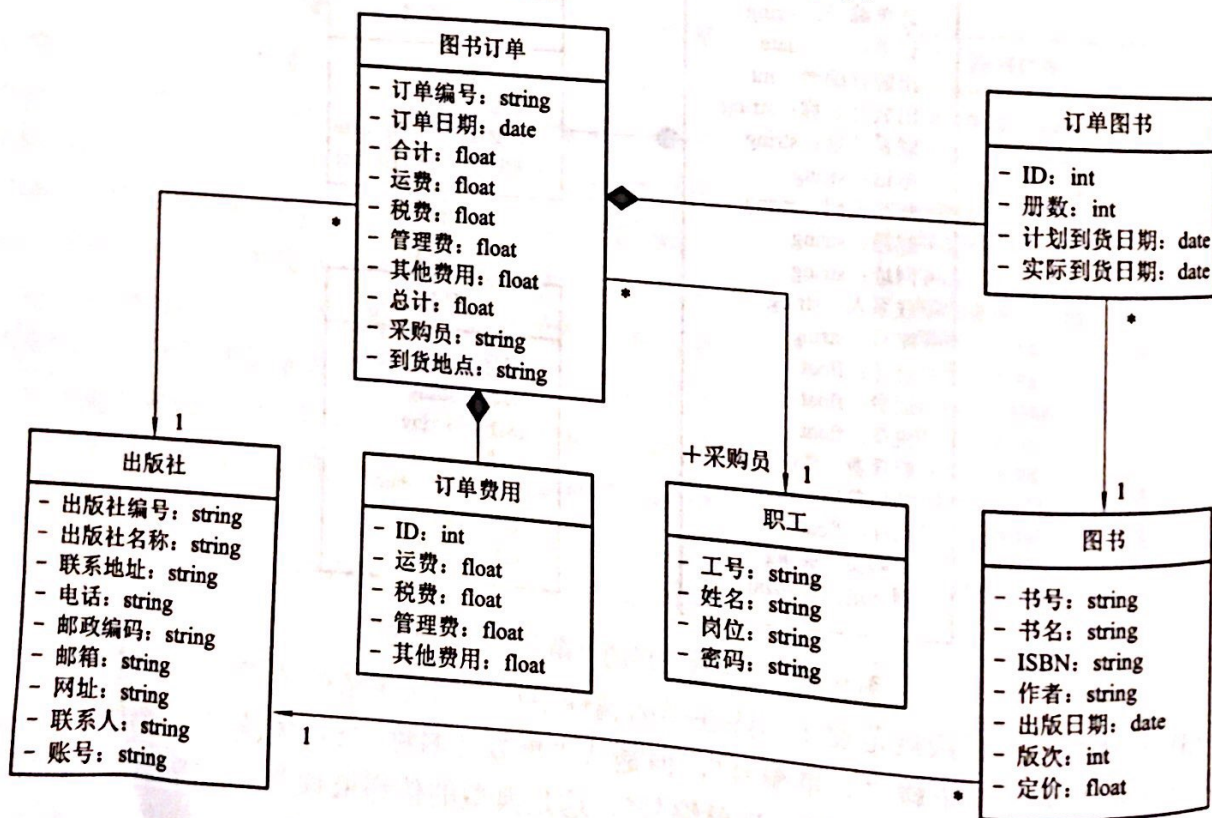


图 9.40 “图书订单”优化类图

对上面这个例子我们可以按事务模式来进行分析。对图书订单而言，它是用来表示订购图书事务的，因此我们首先可以提取订购事务。订购事务的事务细项是订单图书，也就是所订购的“图书”，涉及的人员有“出版社”和“员工”。“图书”是该事务涉及的事物，



这个事务没有地点。如果从订购事务中把订单的费用专门提取出来，按照事务模式就可以构建出图 9.41 所示的类图，该图与图 9.40 所示类图的结构完全一样。这个例子也说明了技术的科学本质是相通的，都能够达到优化结果。

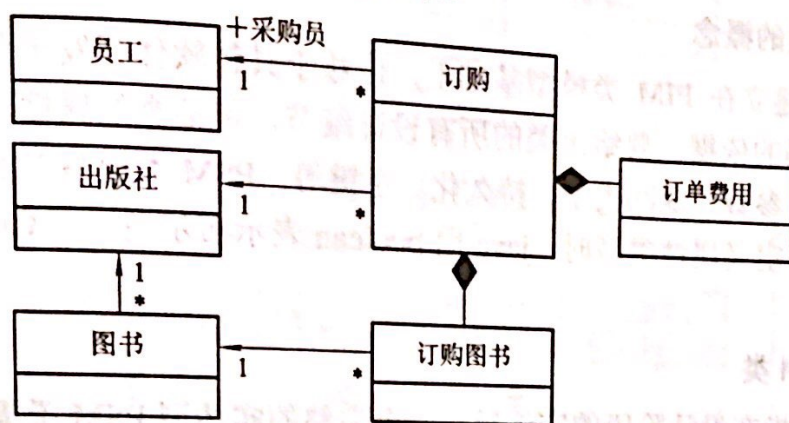


图 9.41 按事务模式构建的“订购”事务的类图

可以采用优化方法对图 9.22 所示的电话订餐系统初步的 PIM 类模型进行优化,得到图 9.42 所示的优化的电话订餐系统 PIM 类模型。在这个图中,除了增加“菜品种类”和“岗位”两个类之外,还表示了由多个菜品构成一份套餐。

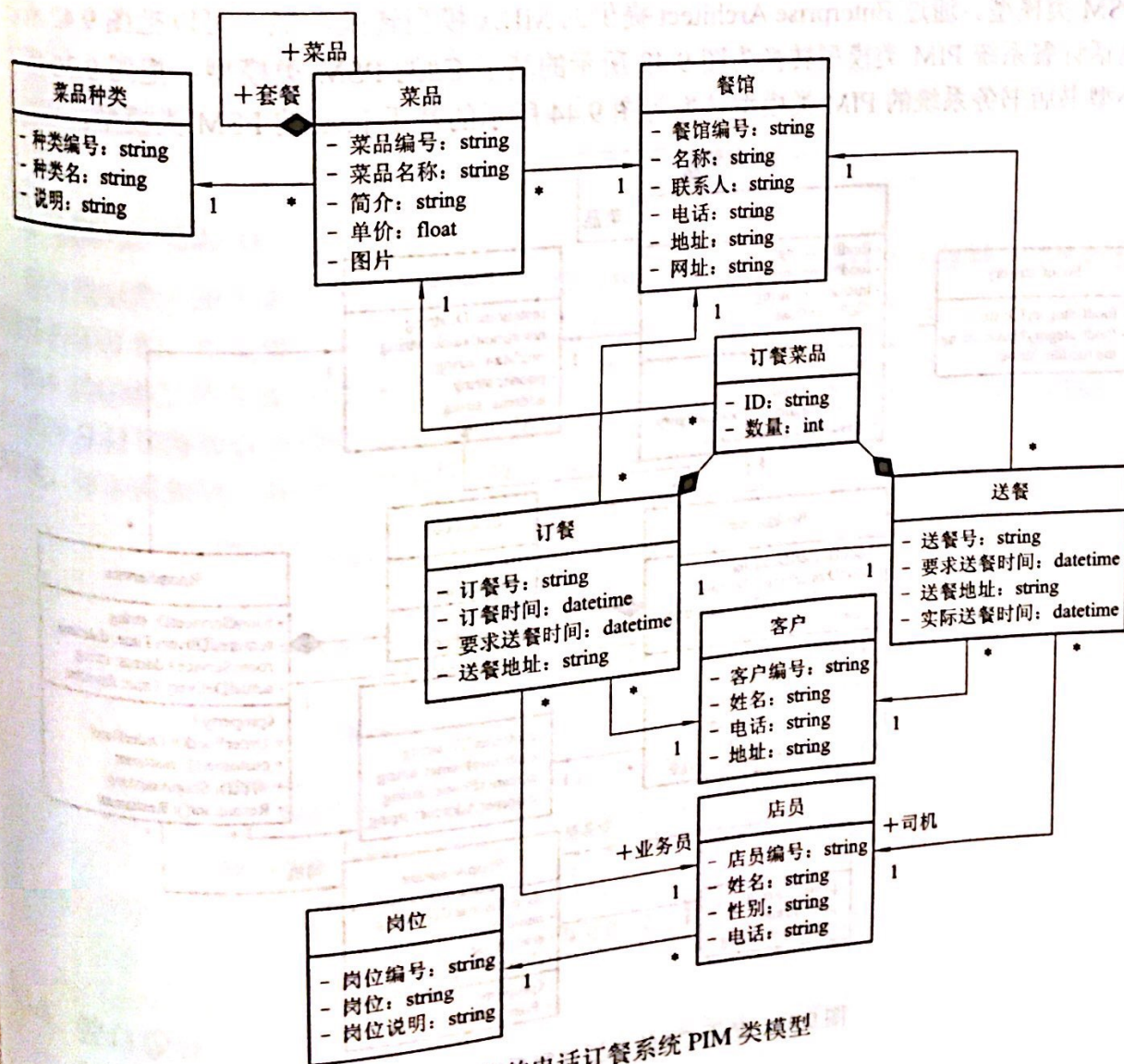


图 9.42 优化的电话订餐系统 PIM 类模型



9.1.5 PSM 类模型

1. PSM 类模型的概念

PSM 类模型是建立在 PIM 类模型基础上，依赖于具体软件开发平台的一组类图。该模型将作为软件编码的依据，要给出类的所有设计细节，包括类的属性、属性的初始值、属性的类型、操作、参数、返回类型、持久化、主键等。PSM 类模型中的类的描述与编程语言有关。例如，在定义属性类型时，java 用 boolean 表示布尔型，而 C# 则用 bool 表示布尔型。

2. 实体型 PSM 类

实体型 PSM 类指在设计阶段的实体类。一个完整的实体型 PSM 类需要具备类的所有性质，具体包括：

- (1) 类属性：包括属性名、可见性、类型，以及多重性和缺省值等。
- (2) 类操作：包括操作名、可见性、操作参数、参数的类型、操作返回类型等。

通过支持 MDA 的建模工具，可以直接把与平台无关的 PIM 类模型转换为与平台相关的 PSM 类模型。通过 Enterprise Architect 提供的 MDA 模型转换功能，可以把图 9.42 所示的电话订餐系统 PIM 类模型转换为图 9.43 所示的基于 C# 的 PSM 类模型；把图 9.30 所示的小型书店书务系统的 PIM 类模型转换为图 9.44 所示的基于 java 的 PSM 类模型。

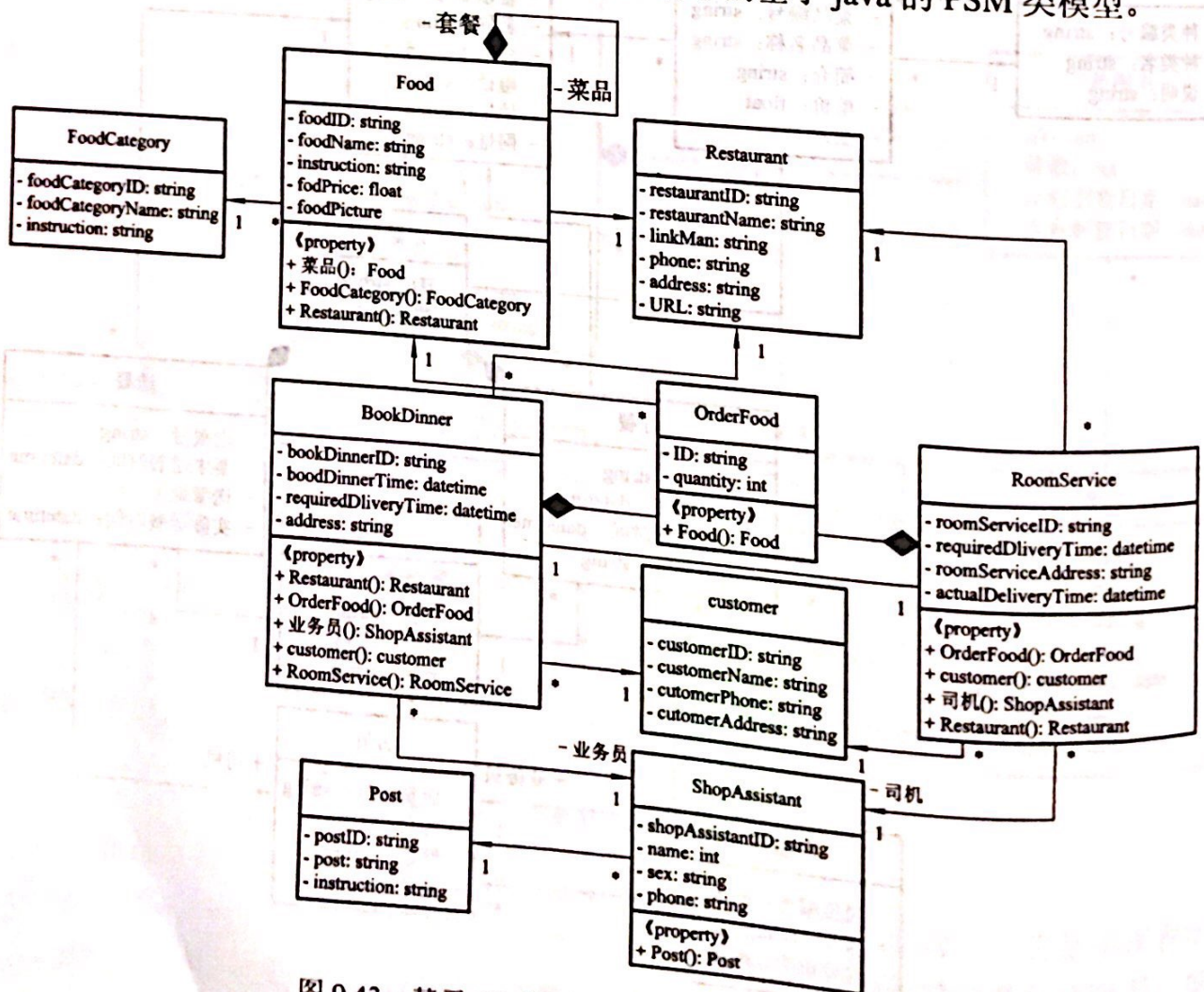
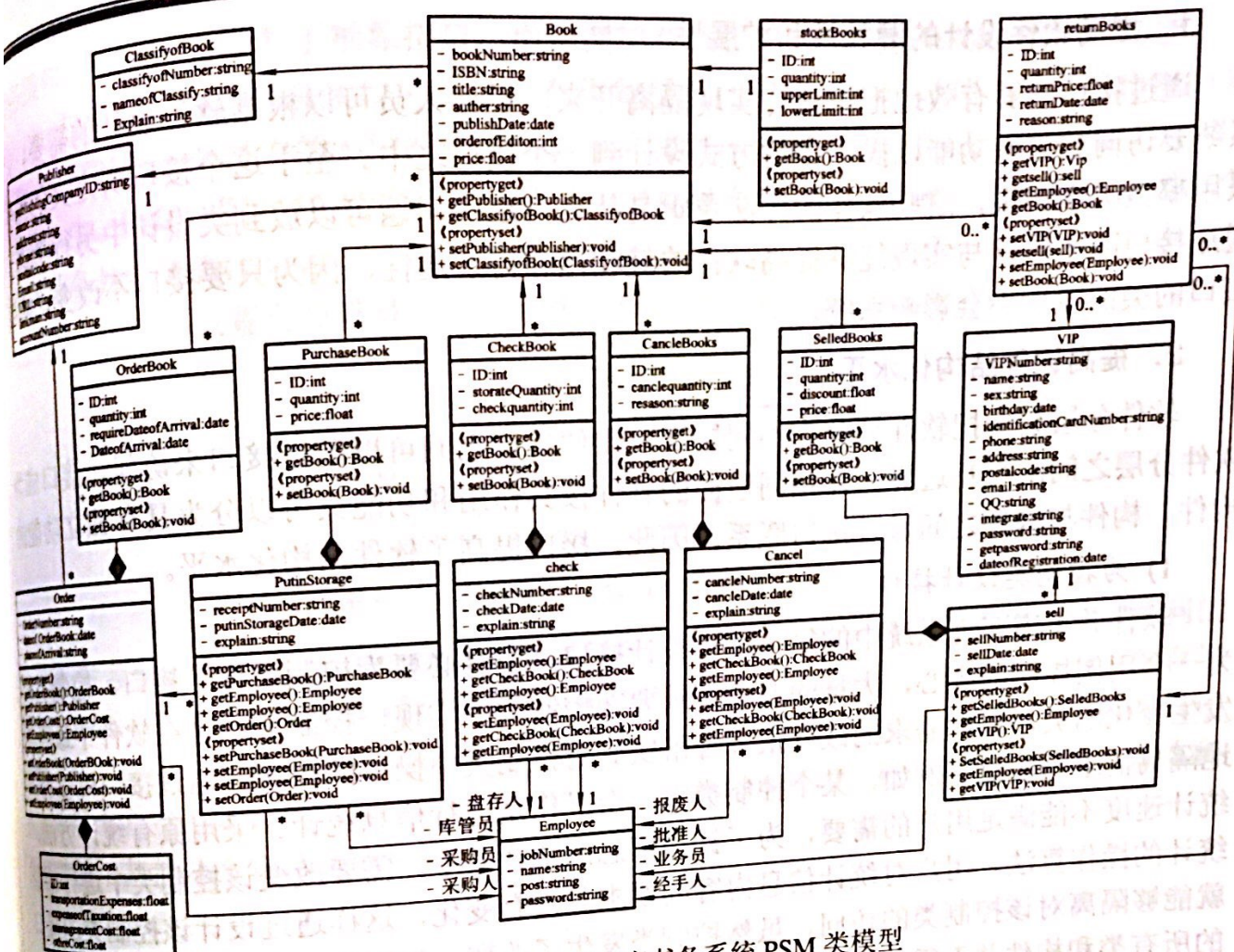


图 9.43 基于 C# 的电话订餐系统 PSM 类模型

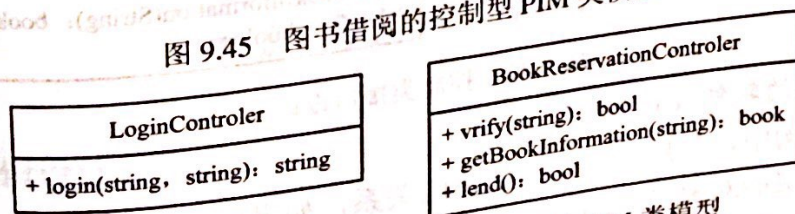
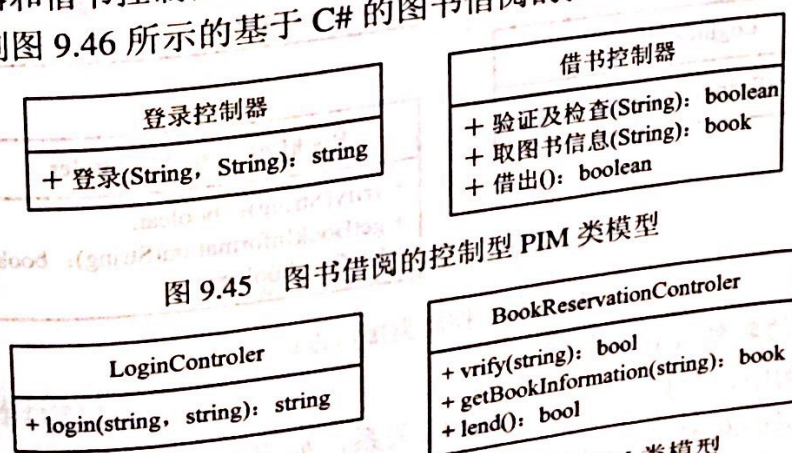




3. 控制型 PSM 类

3. 控制型 PSM 类

因为控制类一般不涉及属性，也不需要持久化，因此控制型 PSM 类的建模只需要给出操作的操作名、可见性、操作参数、参数的类型以及操作返回类型等信息。可以通过支持 MDA 的建模工具直接转换得到控制型 PSM 类模型。例如，某图书借阅系统中有图 9.45 所示的登录控制器和借书控制器两个控制类，通过 Enterprise Architect 提供的 MDA 模型转换功能，可以得到图 9.46 所示的基于 C# 的图书借阅的控制型 PSM 类模型。



9.1.6 接口设计

接口在软件设计中的作用体现在以下几个方面。



1. 提高软件设计的健壮性和扩展性

通过接口能够有效地把设计与实现隔离开来。设计人员可以根据软件设计的需要,把系统要访问的某些功能以操作集的方式设计到一个接口之中。至于这个接口所定义的操作采用哪些类来实现,这些操作的算法逻辑是什么,这些问题可以放到类设计中另行考虑。通过接口隔离设计与实现能够提高软件的健壮性和可扩展性,因为只要接口不改变,实现接口的类的改变不会影响系统。

2. 提高软件结构化水平

软件分层架构把软件划分为不同层次,软件各层之间可以通过接口来访问,接口成为软件分层之间的交互边界。在相同层面的软件按其作用和功能又可以分为具有独立功能的构件,构件与构件之间通过接口联系。因此,接口提高了软件结构化水平。

1) 为控制类设计接口

软件并不需要为系统中的每一个类设计接口,但有必要为控制类设计接口。控制类作为一个用例控制的中心,大量的业务逻辑要在控制类中实现。控制类也是在软件中最容易发生变化的类,软件需求的改变很多都落实到对控制类中操作的改变。通过接口可以有效地隔离需求的变化。例如,某个控制类中一个操作是进行信息统计。采用原有统计方法,统计速度不能满足用户的需要,为了提高信息的统计效率,需要改变该控制类中进行信息统计的操作算法。用户对统计信息内容的要求并没有变化,这样通过设计该控制类的接口就能够隔离对该控制类的访问,虽然控制类发生了改变,但因为接口没有改变,访问接口的所有类和构件并不需要改变。图 9.47 是我们为图 9.46 的登录控制器 LoginControler 和借书控制器 BookReservationControler 两个控制类设计的两个接口。

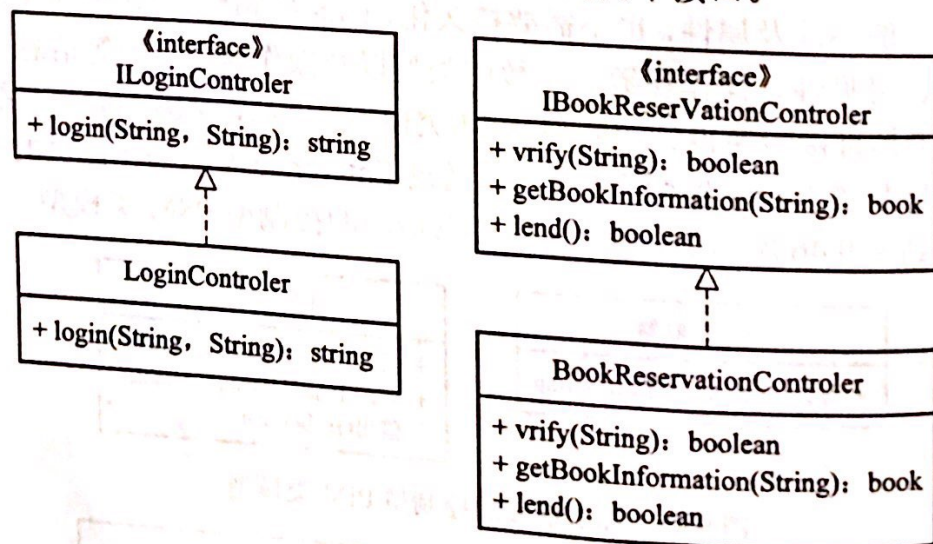


图 9.47 控制类接口设计

2) 为不同层面的软件设计接口

在分层软件架构中,各层之间存在密切的关系,如果由上层中的对象直接访问下层中的对象,会使得各层的对象之间的关系变得错综复杂。为了简化各层之间的访问,可以在每一层上建立提供上层访问的统一的抽象接口,上层对象只能通过该层提供的接口访问下层,不能直接访问下层中的对象。下面我们介绍三种建立抽象接口的方法。

(1) 覆盖法。所谓覆盖法,是指所设计的接口需要覆盖软件在该层所能够提供的



服务操作，不能因为提取了抽象接口，而遗漏掉在该层中某些类能够提供的操作功能。

(2) 归类法。为了提高接口清晰性，在为每层设计接口时，可以使得一个接口对应多个类，也可以使一个类中的不同操作出现在不同接口中。把一组相同或相近的操作放到同一个接口中。例如，图 9.48 描述了在业务逻辑层提供会员管理、订单管理和销售管理三个方面的业务处理逻辑，可以在业务逻辑层设置会员管理接口、订单管理接口和销售管理接口三个接口，分别放置会员管理、订单管理和销售管理的所对应的操作。

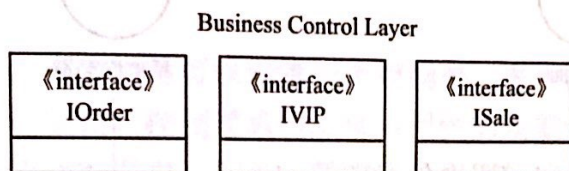


图 9.48 提供订单、会员和销售三个接口

(3) 抽象法。抽象法是指对该层对象的行为进行抽象，提取共性行为作为共同访问的接口。例如，在实体层中无论哪个实体类，都可能提供增加、删除、修改、查询、保存等功能，可以设计一个提供增加、修改、删除、查询、保存操作的接口。例如，图 9.49 描述在实体层中有三个实体类，建立一个统一的 IadmControl 接口，提供增加、删除、修改和查询功能，其他几个实体类实现该接口。

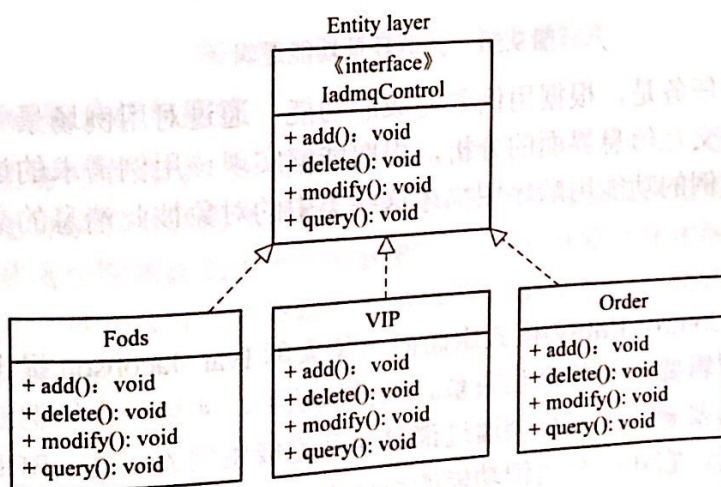


图 9.49 抽象统一接口

9.2 功能逻辑设计

9.2.1 功能逻辑设计的意义

在需求分析中，通过用例来反映信息系统能够给用户提供的功能服务。从系统角度看，需要分析为了实现用例所规定的功能，会涉及哪些系统要素，这些要素是如何通过相互交互来完成用例的功能。从面向对象方法看来，对象是系统的基本构成要素，因此功能逻辑设计就是要分析为完成一个用例的功能，需要涉及哪些对象，这些对象是如何通过相互交互来协作完成用例的功能，并给出其功能逻辑模型的。可以采用 UML 中定义的顺序图来



描述为实现一个用例的功能所涉及的相关对象的交互过程。例如，系统“登录”用例涉及登录界面、登录控制器、会员三个类，三个类中的对象在登录中分别承担接收信息、控制登录过程和信息存储的作用。图 9.50 的顺序图描述为了实现“登录”用例的功能，登录界面、登录控制器和会员三个对象之间的消息交互过程。

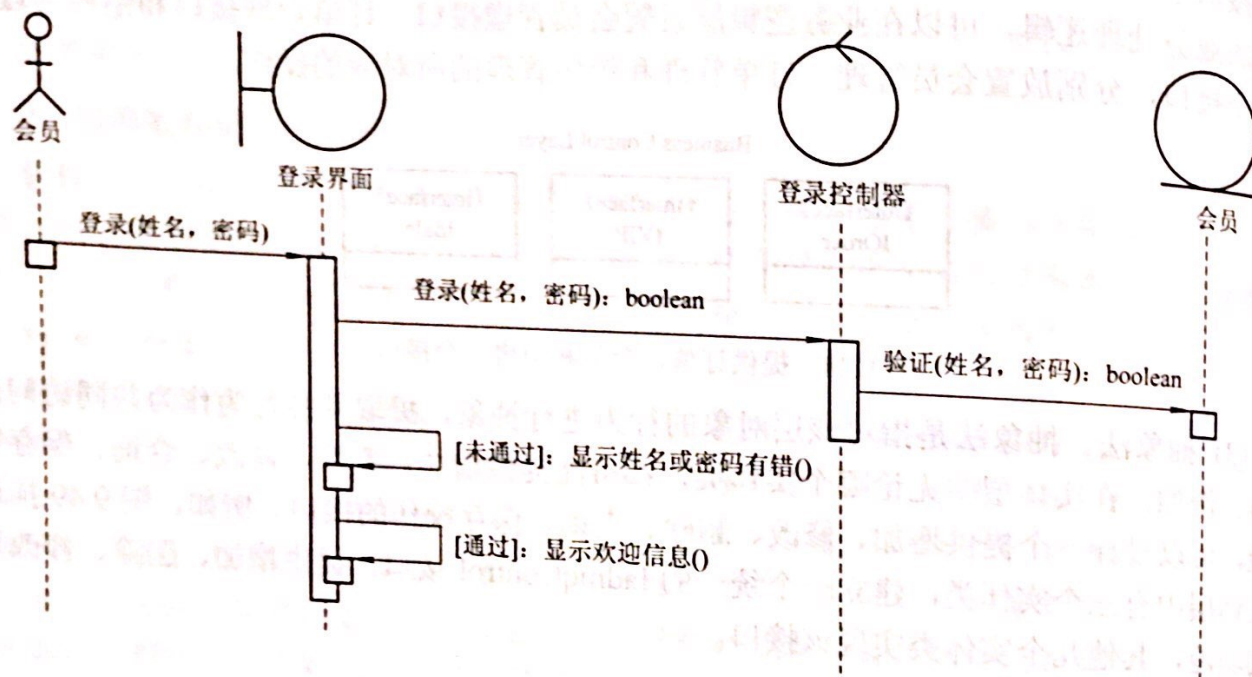


图 9.50 会员登录功能逻辑

功能逻辑设计的任务是，根据用例要完成的功能，通过对用例场景中的用例叙述、用例的执行流程、用例交互信息界面的分析，提取能够实现该用例需求的边界类、控制类和实体类，为了完成用例的功能用顺序图描述这些类中的对象彼此消息的交互过程。

9.2.2 BCE 模式

BCE(Boundary-Control-Entity)模式由面向对象大师 Ivar Jacobson 提出，用来描述实现一个用例的必要的逻辑要素及其交互关系。该模式认为，实现一个用例涉及边界类、控制类和实体类三种逻辑要素，这三种类通过消息交互完成用例的功能。BCE 模式与 MVC 模式有异曲同工的作用，它在一个用例功能的设计中，把信息交互、业务逻辑和信息实体分别由边界类、控制类和实体类承担。边界类的作用是：第一，作为系统与外界的交互接口；第二，承担用例执行期间信息的输入和输出；第三，展示用例功能和相关信息；第四，响应用户所激发的事件。控制类承担用例执行期间用例功能逻辑的实现，控制类是 BCE 模式的核心。在控制类中，通过类的方法来实现需要完成的功能逻辑，一般为一个用例设置一个控制类，但复杂的用例可以设置多个控制类。实体类是用例所涉及的 PIM 类模型中的类，对应着业务领域中的业务实体。图 9.50 是通过 BCE 模式描述的会员登录功能逻辑设计。

9.2.3 功能逻辑设计

1. 分析用例的 BCE 类

实现一个用例需要 BCE 三种类型的类，需要根据用例的职责，分析这三种类型的类。下面我们分别介绍这三种类。



1) 边界类

一个用例一般涉及一个边界类。例如，登录界面就是学生登录用例的边界类。对于存在人机交互界面的用例，边界类就表示展现该用例功能，以及输入输出信息的界面；对于直接与外部系统或外部设备联系的用户，边界类则是用例与外部系统或外部设备的接口。需要注意的是，在用例功能逻辑设计中，边界类是负责用例信息的交互接口的一个抽象单元，实际上在功能逻辑模型中一个边界类常常对应着实现阶段的多个具体的边界对象。

2) 控制类

根据用例的复杂度，一个用例可以设置一个控制类、多个控制类，或者不设控制类，但较常见的是设置一个控制类。控制类承担实现该用例的功能逻辑，并由控制类中的对象的若干操作来实现用例的功能逻辑。例如，学生登录用例的控制类是登录控制器。

3) 实体类

用例所涉及的实体类一般在 PIM 类模型之中。例如，学生登录用例涉及学生类。借书用例涉及的三种类型的类见图 9.51。

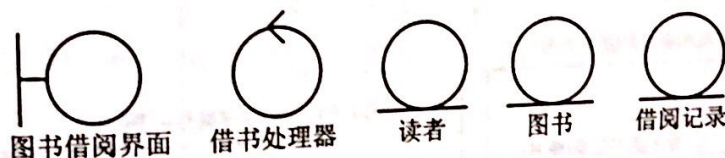


图 9.51 借书用例的三种类型的类

2. 用例功能逻辑分析与建模

一个用例的功能可以通过一组类中的对象在执行过程中通过交互消息来完成，用顺序图能够恰当地描述对象的消息交互过程。对用例分析的主要依据是用例场景中的用例叙述中的事件流，以及活动图描述的用例执行流程。用例的对象交互体现为由多个类中的对象协作完成一项任务，对象通过互相发送消息来实现对象交互，对象之间发送的消息除了一般信息之外，大多是调用另外一个对象的一个操作。

用例对象交互过程应该与用例叙述中事件流的顺序相一致。图 9.52 描述借书用例的对象交互过程。按照图 7.22 中借书用例的事件流，首先由图书管理员扫描读者的借书证号，并把借书证号提交给图书借阅界面。系统通过图书借阅界面接收到借书证号之后，需要对接收到的借书证的合法性进行验证。如果借书证合法，还要检查读者是否存在超借或超期未归还图书的问题。如若存在这些问题，则系统给出提示，本次不能借书。在借书控制器中设置验证及检查操作来验证借书证的合法性，并检查是否存在超借和超期的问题。图书借阅界面把接收到的借书证号作为“验证及检查”操作的参数，给借书控制器发送一条“验证及检查(借书证号)”的消息，通过这个消息调用借书控制器中的“验证及检查”操作。借书控制器为了验证借书证的合法性，给读者类发送一个“检查借书证(借书证号)”的消息，读者类中的这个操作将在读者类中检查是否存在给定借书证号的读者，如果存在说明该借书证是合法的，否则不合法。“检查借书证(借书证号)”操作返回一个 boolean 型值，该值为真，表示借书证合法，为假表示借书证不合法。如果不合法，则在图书借阅界面显示该借书证不合法的提示信息。在检查了借书证的合法性之后，借书控制器通过给借阅记录类发送“检查超期超借(借书证号)”的消息，由借阅记录检查持该借书证的读者是否存在



超期或超借的情况, 如果存在则操作返回假值, 系统在图书借阅界面显示存在超期超借的提示信息。

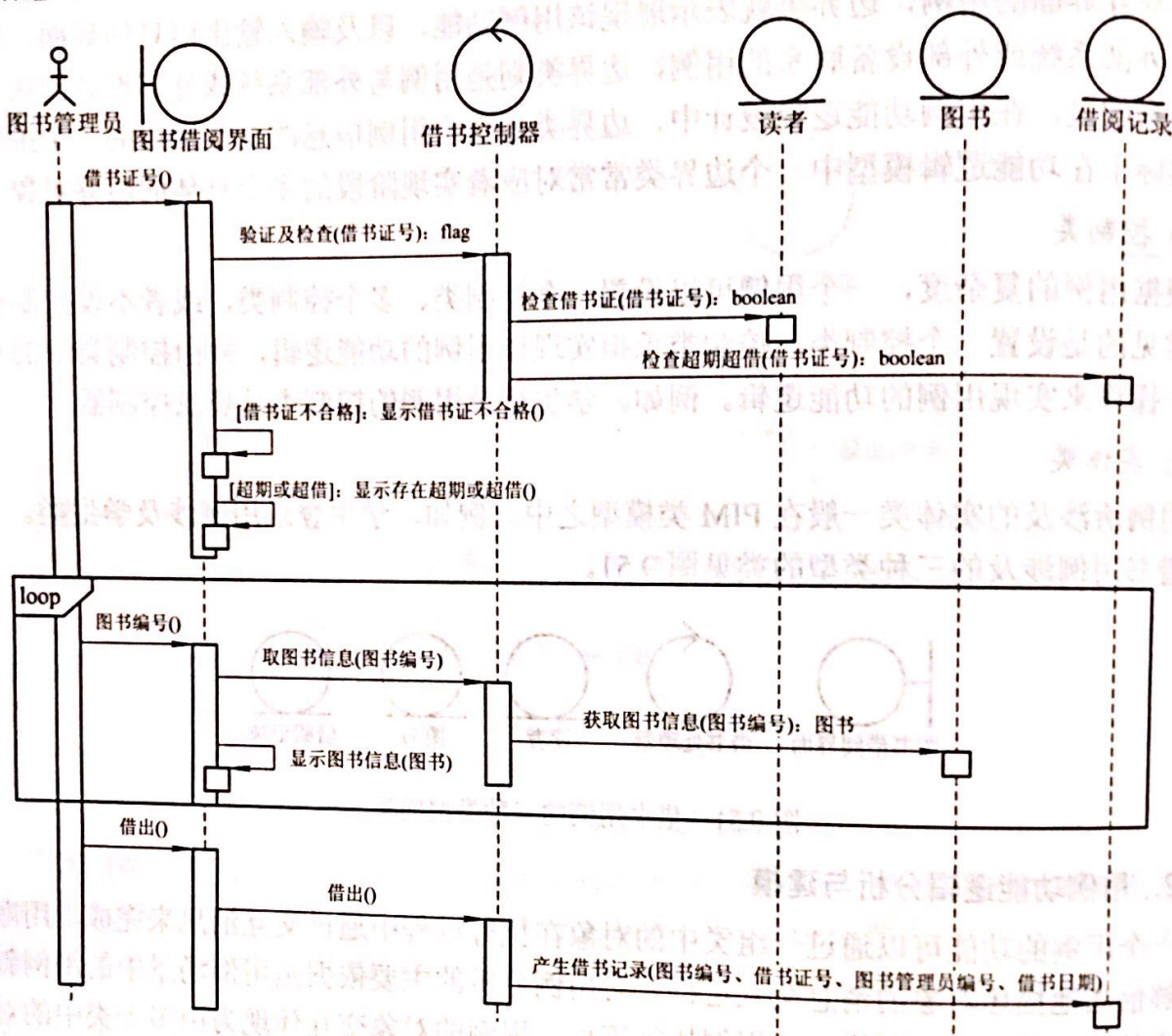


图 9.52 借书用例功能逻辑设计模型

如果借书证合法, 并且也不存在超期或超借的问题, 图书管理员将把读者本次要借的图书的图书编号扫描输入给系统。图书借阅界面接收到扫描的一个图书编号后, 先给借书控制器发送“取图书信息(图书编号)”消息, 图书控制器通过给读者类发送“获取图书信息(图书编号)”消息, 从读者类中获取该图书的图书信息, 并把该图书信息显示在图书借阅界面上。如果本次读者借阅多本图书, 可以重复这个过程。

本次要借阅的图书扫描完后, 图书管理员点击图书借阅界面上的“借出”按钮, 启动借书控制器的“借出”操作, 在借阅记录类中产生本次借阅记录。

9.3 数据库设计

9.3.1 概述

数据库设计是指根据业务需求、信息需求和处理需求, 确定信息系统中的数据库结构、数据操作和数据一致性约束的过程。数据库设计的基本过程可分为需求分析、概念设计、逻辑设计和物理设计四个步骤。需求分析的主要工作是调查和分析用户的业务活动、信息

